课程笔记：从零用纯 PyTorch 构建完整大语言模型

整理：Bowen-mantou

2026 年 10 月 23 日

视频作者/频道：AIDeepCoder（B 站）
发布日期：2026-09-27
视频时长：6:06:22
视频链接：https://www.bilibili.com/video/BV1VJa869ECR
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 Part 0 —课程导览与目标 **2**

1.1 为什么要有这门课：从「会用」到「会建」 2
1.2 课程目标：把 LLM 的完整生命周期走一遍 2
1.3 学习路线与代码结构 2
1.4 环境与前置要求 3
1.5 本章小结 4

2 Part 1 —核心 Transformer 架构 **4**

2.1 先把镜头拉远：GPT 训练流程与 Transformer 的位置 4
2.2 位置编码：为什么需要、正弦公式怎么来的 5
2.2.1 问题：注意力本身没有顺序感 5
2.2.2 正弦公式 5
2.2.3 实现细节：register_buffer 与奇偶维填充 5
2.3 Self-attention 的手算：一个小例子 5
2.3.1 四步计算 6
2.3.2 为什么必须除以 $\sqrt{d_k}$ 6
2.4 单头注意力：把公式变成 nn.Module 6
2.5 多头注意力：拆分、并行、拼接、投影 7
2.5.1 形状的完整旅程（本节最该记住的东西） 7
2.5.2 两个实现技巧 7
2.6 因果掩码：让模型看不见未来 8
2.7 前馈网络：4 倍扩张 + GELU 9
2.8 残差连接与 LayerNorm：让深网络能训练 9
2.8.1 Pre-norm：先归一化再进子层 10
2.9 把一切堆起来：Transformer block 与 GPT 10
2.9.1 从 block 到模型 10
2.9.2 把 Part 1 的零件组装起来 11
2.10 本章小结 11

3 Part 2 —训练一个小 LLM **12**

3.1 本讲定位：把 Part 1 的模型真正训起来 12
3.2 字节级分词器：每个字节就是一个 token 12
3.3 数据集与批采样：随机窗口 + label shift 13
3.4 模型与交叉熵：位置、残差与损失 14
3.5 训练循环：AMP、梯度裁剪、验证与检查点 14
3.6 采样：温度、top-k、top-p 与自回归生成 15
3.7 实测：损失在降，但模型仍然“不会说话” 16
3.8 本章小结 17

4 Part 3 —架构现代化 **17**

4.1 本讲定位：把“2017 年的 Transformer”升级成现代架构 17
4.2 RMSNorm：只除均方根，不减均值 18

4.3 SwiGLU：用门控 + SiLU 替换 FFN 激活 . . . 18
4.4 KV Cache：用显存换计算 . . . 18
4.5 Grouped-Query Attention：在“效果”和“缓存大小”之间折中 . . . 19
4.6 滑动窗口注意力与 Attention Sink . . . 19
4.7 位置编码的进化：绝对 → 相对 → RoPE . . . 19
4.8 代码走读：现代 GPT 与带缓存生成 . . . 20
4.9 本章小结 . . . 21

5 Part 4 —扩展规模 21
5.1 本讲定位：从「跑通训练」到「生产级训练」 . . . 21
5.2 train.py 的参数面：一次训练需要哪些「旋钮」 . . . 22
5.3 分词器升级：从字节级到 BPE . . . 22
5.3.1 为什么字节级不够 . . . 22
5.3.2 BPE 的机制：反复合并最频繁的相邻对 . . . 22
5.4 数据加载与模型配置的重建 . . . 23
5.5 学习率计划：warmup + 余弦衰减 . . . 23
5.6 混合精度与梯度累积 . . . 24
5.6.1 混合精度：用更低的数值精度换吞吐 . . . 24
5.6.2 梯度累积：用显存换「等效大 batch」 . . . 24
5.7 检查点、原子保存与断点续训 . . . 25
5.7.1 检查点里要存什么 . . . 25
5.7.2 SIGINT/SIGTERM：按 Ctrl+C 也要保住当前进度 . . . 25
5.8 日志与可观测性：训练到底健康不健康 . . . 25
5.9 本章小结 . . . 26

6 Part 5 —Mixture-of-Experts 26
6.1 本讲定位：把 FFN 也「现代化」 . . . 26
6.2 核心思想：稀疏激活换参数容量 . . . 26
6.3 参考实现：Switch Transformer 的简化版 . . . 27
6.4 组件一：专家 MLP . . . 27
6.5 组件二：Top-K 门控（gating） . . . 27
6.6 组件三：MoE 层的前向（token 展平与按专家填回） . . . 28
6.7 负载均衡：为什么需要辅助损失 . . . 28
6.7.1 问题：专家「名人化」 . . . 28
6.7.2 机制：importance × load 的辅助损失 . . . 28
6.8 与稠密层混合：hybrid 架构 . . . 29
6.9 范围说明：分布式与训练-推理差异不在本讲 . . . 29
6.10 本章小结 . . . 29

7 Part 6 —监督微调（SFT） 30
7.1 6.1 工程入口：从基座 checkpoint 出发 . . . 30
7.2 6.2 指令数据：从 instruction/input/output 到 (prompt, response) . . . 30
7.3 6.3 对话模板：把 (prompt, response) 变成一段文本 . . . 31
7.4 6.4 批处理与 loss mask：只让回答参与损失 . . . 31

7.5 6.5 训练循环：前向、反向与工程细节 . . . 32
7.6 6.6 采样评测：提前停止与 KV cache 复用 . . . 33
7.7 6.7 从玩具数据到规模化：过拟合与泛化边界 . . . 33
7.8 本章小结 . . . 33

8 Part 7 —奖励建模 34
8.1 7.1 为什么需要奖励模型 . . . 34
8.2 7.2 偏好数据的结构与加载 . . . 34
8.3 7.3 奖励模型架构：encoder-only Transformer + scalar head . . . 34
8.4 7.4 前向：padding mask 与 masked mean pooling . . . 35
8.5 7.5 Bradley-Terry 损失与边际排序损失 . . . 36
8.6 7.6 训练循环与评测：小数据只能验证管线 . . . 36
8.7 7.7 与 Part 8 的衔接：进入强化学习 . . . 37
8.8 本章小结 . . . 37

9 Part 8 —基于 PPO 的 RLHF 37
9.1 RLHF 三阶段与 PPO 的位置 . . . 38
9.2 把语言生成写成强化学习问题 . . . 38
9.3 训练器结构：三份权重与超参数 . . . 39
9.4 Rollout：采样、奖励与 token 级数据 . . . 40
9.5 从 token 到 log 概率：shift 标签与 gather . . . 41
9.6 奖励塑形、KL 控制与优势估计 . . . 42
9.7 PPO 目标与损失函数 . . . 43
9.8 训练与评测：日志、KL 变化与 demo 的局限 . . . 44
9.9 本章小结 . . . 45

10 总结与延伸 46
10.1 五个最值得带走的结论 . . . 46
10.2 把课程接回你的项目与面试 . . . 46
10.3 如果继续深入：三条延伸线 . . . 47

1 Part 0 —课程导览与目标

这是一门「用纯 PyTorch 从零构建完整大语言模型」的实战课程。它的定位不是再讲一遍 Transformer 论文，也不是教人调 API，而是把现代 LLM 的**完整生命周期**拆成可运行的代码：从最底层的注意力机制开始，一路走到预训练、监督微调、奖励建模与 RLHF。讲师在开场就把课程目标说得很直接——学完这门课，你不仅知道怎么「用」一个模型，还会理解让 GPT 这类模型真正工作的每一个核心构件，并且亲手实现它们。

1.1 为什么要有这门课：从「会用」到「会建」

市面上关于 LLM 的教程大致分两类：一类教你用现成模型（提示词、API、应用编排），另一类展示别人是怎么构建的。讲师点名感谢了 Sebastian Raschka 等人的公开教程，认为它们是自己理解 LLM 原理的灵感来源；但本课程想做一件这些教程没有集中做的事：**在一个地方、用一套连续的代码，把现代 LLM 的端到端工作原理完整展示出来**。

这种「端到端」有两层含义。第一是**横跨整条流水线**：预训练、SFT、对齐（奖励模型 + RLHF）都在课程里实现。第二是**纵跨抽象层级**：从「注意力矩阵长什么样」到「怎样在笔记本上把一个玩具模型训起来」，中间没有断点。为此课程在理论深度上刻意克制——用讲师的话说，**只使用推进到实操所必需的最小理论量**（“the minimum necessary force”），把大部分篇幅留给代码与调试。

1.2 课程目标：把 LLM 的完整生命周期走一遍

讲师把 GPT 助手的训练流程概括为四个阶段，这四个阶段构成了整门课的主线：

1. **预训练（pretraining）**：把互联网规模的数据处理成数万亿 token，用成千上万块 GPU 训练数月，得到一个「基座模型」（base model）。这是最贵的一步，也是本课程 Part 2–Part 5 的主题。
2. **监督微调（SFT, supervised fine-tuning）**：用相对少量的指令—回答数据，把「只会预测下一个词」的基座模型变成「会按指令作答」的助手。对应 Part 6。
3. **对齐（alignment）**：让模型不输出有偏见、不安全或不符合人类偏好的内容。课程把它拆成两步：先训练奖励模型（Part 7），再用 RLHF 优化策略（Part 8）。
4. **部署与推理**：课程在 Part 4 讨论规模扩展、混合精度、检查点与日志，为「真的跑起来」补上工程细节。

这门课要交付的能力

学完并跟着敲完代码后，你应该能够：(1) 从零写出可训练的 decoder-only Transformer；(2) 在单机 CPU/GPU 上训练一个字符级小模型并采样；(3) 把原生 Transformer 现代化（RMSNorm、RoPE、SwiGLU、KV cache 等，Part 3）；(4) 完成 SFT、奖励模型与 PPO-RLHF 的最小闭环；(5) 理解 MoE 这类大模型扩展技术的基本机制。

1.3 学习路线与代码结构

课程共 9 个部分（Part 0 是导览，Part 1–Part 8 是 8 个技术模块），每部分对应仓库里的一个目录：

部分	主题	主要内容
Part 1	核心 Transformer 架构	位置编码、self-attention、单头/多头注意力、FFN、残差 + LayerNorm、Transformer block
Part 2	训练一个微型 LLM	字节级分词、数据集与批次、交叉熵损失、训练循环、温度/top-k/top-p 采样、验证集评估
Part 3	现代化架构	RMSNorm、RoPE、SwiGLU、滑动窗口注意力、KV cache、滚动缓冲
Part 4	扩大规模	梯度累积、混合精度（AMP）、检查点、日志、学习率调度
Part 5	混合专家模型（MoE）	路由与门控、负载均衡、稳定性技巧、稠密与 MoE 混合块
Part 6	监督微调	指令数据集、prompt/response 格式化、掩码、SFT 评估
Part 7	奖励建模	成对偏好数据、Bradley–Terry 损失、排序损失
Part 8	RLHF（PPO）	价值头、rollout、奖励与 KL 惩罚、GAE、PPO 裁剪目标

仓库的代码组织有一个统一约定，值得先记住：**每个部分都有自己的 `orchestrator.py` 作为入口**。它把该部分的演示、测试与可视化按顺序串起来，你从入口往下读，就能依次进入各个模块。Part 1 的目录里，模块与课程小节的对应关系是：

- `pos_encoding.py`：位置编码（绝对可学习 + 正弦两种）；
- `attn_numpy_demo.py`：用 NumPy 和极小数字手算一遍 self-attention；
- `single_head.py`：单头自注意力的 PyTorch 实现；
- `multi_head.py`：多头自注意力（带 shape tracing）；
- `ffn.py`：前馈网络（GELU，隐层宽度 $4 \times d_{\text{model}}$）；
- `block.py`：残差 + LayerNorm 组成的 Transformer block；
- `attn_mask.py`：因果掩码工具；`vis_utils.py`/`demo_*.py`：可视化与形状演示；`tests/`：单元测试。

1.4 环境与前置要求

环境要求被刻意压到最低：**一台装了 Python 和 PyTorch 的笔记本**即可；如果有 GPU 可以装 CUDA，但课程里的所有内容都被设计成**在 CPU 上也能跑**，只是会慢一点。仓库根目录提供 `requirements.txt`，Part 1 的依赖基本只有 PyTorch、NumPy 与 matplotlib（用于画注意力热图）。

课程不假设你有分布式训练或编译器背景，但在读 Part 1 之前，最好对「矩阵乘法、softmax、线性层」有直觉——如果没有，讲师建议先在 NumPy 里把注意力公式手推一遍（Part 1 正好提供了这样的 NumPy demo）。

1.5 本章小结

- 课程定位：用一套连续的纯 PyTorch 代码，端到端展示现代 LLM 的完整生命周期，理论只保留推进实操所必需的部分。
- 主线四阶段：预训练 → SFT → 对齐（奖励模型 + RLHF）→ 部署/推理；8 个 Part 与之对应。
- 代码约定：每个部分以 `orchestrator.py` 为入口，模块与小节一一对应；Part 1 的文件名就是最好的目录。
- 环境：笔记本 + Python + PyTorch 即可，CPU 可跑；GPU 可选。

2 Part 1 —核心 Transformer 架构

Part 1 是整个课程的地基：把 2017 年论文《Attention Is All You Need》里的原生 Transformer，用代码一个组件一个组件地实现出来。讲师的处理方式很工程化——先给你一张「Transformer 在整个训练流水线里处于什么位置」的全景图，然后按数据流动的顺序逐个实现：位置编码 → self-attention → 单头 → 多头 → 因果掩码 → 前馈网络 → 残差与 LayerNorm → 堆叠成 block。最后你会得到一个可以前向传播的 GPT 模型，为 Part 2 的训练做好准备。

2.1 先把镜头拉远：GPT 训练流程与 Transformer 的位置

讲师先回到 Part 0 提到的四阶段流水线，指出**我们今天只关心第一步里最核心的那块**：原生 Transformer。它来自 2017 年的论文，原本是编码器—解码器结构；但现代 LLM 几乎都是 **decoder-only**（只用解码器），所以课程只实现解码器一侧。

一次前向传播的数据流可以写成下面的顺序，后面每一小节都会对应其中一段：

1. 文本先被**分词**（tokenization），每个 token 变成一个整数 ID；
2. ID 进入**词嵌入**（token embedding），得到 (B, T, d_{model}) 的张量；
3. 加上**位置编码**（positional encoding），注入顺序信息；
4. 通过一个线性层得到 **Q**、**K**、**V** 三份拷贝，再做**缩放点积注意力**；
5. decoder-only 架构在 softmax 之前加**因果掩码**（causal mask），让每个位置只能看到自己和左边；
6. 注意力输出与输入做**残差相加**，再过 **LayerNorm**；
7. 进入**前馈网络**（FFN），同样残差 + LayerNorm；
8. 以上构成一个 **Transformer block**，堆叠 N 层；
9. 最后接一个线性头得到词表大小的 logits，softmax 后就是下一个 token 的概率分布。

一句话抓住 Transformer

Transformer block =「多头自注意力 + 前馈网络」，每个子层都包在「残差连接 + LayerNorm」里；把它堆 N 次，再接一个线性分类头，就是 GPT 的主干。Part 1 剩下的内容，就是把这句话里的每个名词变成可运行的代码。

2.2 位置编码：为什么需要、正弦公式怎么来的

2.2.1 问题：注意力本身没有顺序感

Self-attention 的计算是「每个 token 与所有 token 做加权求和」，它**对输入顺序是置换不变（permutation-invariant）的**：把句子里的词打乱，注意力算出来的结果只是跟着换位置，模型本身不会「知道」谁在前谁在后。而语言的意义高度依赖顺序（“The cat sat on the mat” 与 “The mat sat on the cat” 完全不同）。因此必须显式地把位置信息注入到输入里。

课程实现了两种经典的**绝对位置编码**（absolute positional encoding）：

1. **可学习位置编码**（learned）：给每个位置 $0, 1, \ldots, T-1$ 分配一个可训练向量，直接加到词嵌入上；
2. **正弦位置编码**（sinusoidal）：用不同频率的正弦/余弦函数解析地生成位置向量，不参与训练——这是 2017 年原论文采用的方法，也是 Part 1 的重点。

2.2.2 正弦公式

设位置为 pos、维度下标为 i、模型维度为 d_{model}，原论文的编码是：

$$PE_{(\text{pos},\,2i)} = \sin\Big(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\Big), \qquad PE_{(\text{pos},\,2i+1)} = \cos\Big(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\Big).$$

其中：

- pos：token 在序列中的位置（0 到 $T-1$）；
- i：维度下标；偶数维用 sin，奇数维用 cos；
- $10000^{2i/d_{\text{model}}}$：频率分母。$i$ 越大，分母越大、频率越低，于是不同维度覆盖了从高频到低频的一组「波长」；
- 输出 $PE \in \mathbb{R}^{T \times d_{\text{model}}}$，与词嵌入逐元素相加。

为什么用这种设计？论文给出的直觉是：对任意固定偏移 k，$PE_{\text{pos}+k}$ 可以表示成 PE_{pos} 的线性函数，因此模型容易学会「相对位置」；而且它不依赖训练长度，理论上可以外推到比训练更长的序列（是否真的外推得好是另一个问题，Part 3 会给出更好的答案 RoPE）。

2.2.3 实现细节：register_buffer 与奇偶维填充

代码里有两个容易忽略但很重要的细节。第一，正弦编码是**固定常数**，所以用 `register_buffer` 注册而不是 `nn.Parameter`——它会跟随模型搬到 GPU、被保存进 state dict，但**不产生梯度**。第二，先算 $\text{div_term} = \exp(-i \cdot \log(10000)/d_{\text{model}})$，再用「$\sin(\text{pos} \cdot \text{div_term})$ 填偶数列、cos 填奇数列」的方式一次性构造完整个矩阵，这比逐位置循环快得多。

2.3 Self-attention 的手算：一个小例子

在写模块之前，讲师先用一个「极小数字」的 NumPy demo 把注意力公式手算一遍，让读者对形状和数值都有直觉。例子刻意缩小：批大小 $B = 1$、序列长度 $T = 3$、模型维度 $d_{\text{model}} = 4$，即输入 X 是 $1 \times 3 \times 4$。

2.3.1 四步计算

1. **投影出 Q、K、V**：$Q = XW_Q$，$K = XW_K$，$V = XW_V$。这里 W_Q, W_K, W_V 是三个线性层（形状 $d_{\text{model}} \times d_k$），把每个 token 的 4 维表示映射到 d_k 维；
2. **算注意力分数**：scores $= QK^\top$，形状变成 $T \times T$——第 i 行第 j 列表示「第 i 个 token 对第 j 个 token 的关注原始分数」；
3. **缩放 + softmax**：weights $= \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$，每行归一化成概率；
4. **加权求和**：output $=$ weights $\cdot V$，形状 (T, d_k)，每个位置得到「按注意力权重混合后的值向量」。

用公式写出来就是：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

其中 Q, K, V 分别由输入经三个线性变换得到，d_k 是每个头的维度，$\sqrt{d_k}$ 是缩放因子。

2.3.2 为什么必须除以 $\sqrt{d_k}$

点积的结果随维度增长而变大：若 q, k 的各分量独立、均值 0、方差 1，则 $q \cdot k$ 的方差约为 d_k。维度一大，分数就会很大，softmax 随之饱和（几乎 one-hot），梯度接近于零，训练会变慢甚至停滞。除以 $\sqrt{d_k}$ 相当于把分数的方差拉回 1 左右，让 softmax 工作在梯度敏感的区间。这个细节在 demo 里也会被打印出来，因为它是注意力实现里最常被漏掉的一步。

2.4 单头注意力：把公式变成 nn.Module

有了手算直觉，就可以把它封装成 PyTorch 模块。`single_head.py` 中的单头自注意力类只做一件事：把一个 (B, T, d_{model}) 的输入映射成 (B, T, d_k) 的注意力输出，并返回注意力权重。

实现上有三个要点：

- **三个独立线性层**：W_Q, W_K, W_V 分别是 `nn.Linear(d_model, d_k, bias=False)`；这与多头里「一次性投影再拆分」的做法不同，单头写法更直观；
- **因果掩码**：在 softmax 之前调用 `causal_mask(T)`，把上三角（未来位置）填成 $-\infty$；
- **形状**：Q, K, V 均为 (B, T, d_k)，scores 为 (B, T, T)，weights 为 (B, T, T)，输出为 (B, T, d_k)。

```python
def forward(self, x):              # x: (B, T, d_model)
    B, T, _ = x.shape
    q, k, v = self.q(x), self.k(x), self.v(x)   # each (B,T,d_k)
    scale = 1.0 / math.sqrt(q.size(-1))
    attn = torch.matmul(q, k.transpose(-2, -1)) * scale
    mask = causal_mask(T, device=x.device)
    attn = attn.masked_fill(mask.squeeze(1), float('-inf'))
    w = F.softmax(attn, dim=-1)
    out = torch.matmul(w, v)                    # (B,T,d_k)
    return out, w
```

Listing 1: 单头自注意力的核心前向（节选自 `part_1/single_head.py`）

这段代码与上一节的手算一一对应，唯一的新东西是掩码（下一小节展开）。讲师还给它写了单元测试：用固定随机种子构造输入，把「NumPy 手算的权重」与「PyTorch 模块算出的权重」逐元素比较，确保两边在数值上一致——这一步对后面复用注意力模块很重要。

2.5 多头注意力：拆分、并行、拼接、投影

单头注意力的表达能力有限：它只有一套 $Q/K/V$ 投影、只能学到一种「关注模式」。多头注意力（multi-head attention, MHA）的思路是**把 d_{model} 维拆成 h 个 d_{head} 维的子空间**，每个头独立做一次注意力，最后把结果拼接、再投影回 d_{model}。这样模型可以同时学习「关注相邻词」「关注句法依存」「关注远距离指代」等不同的关系。

2.5.1 形状的完整旅程（本节最该记住的东西）

课程用一组具体数字把形状走了一遍：$B=1$、$T=5$、$d_{\text{model}}=12$、$h=3$，于是 $d_{\text{head}}=12/3=4$。前向传播中每个张量的形状是：

阶段	形状	说明
输入 x	$(1,5,12)$	(B,T,d_{model})
`qkv = self.qkv(x)`	$(1,5,36)$	一次线性层得到 $3d_{\text{model}}$
`view`(B,T,3,h,d_h)	$(1,5,3,3,4)$	把 36 拆成 $3\times3\times4$
`unbind(dim=2)`	3 个 $(1,5,3,4)$	得到独立的 q,k,v
`transpose(1,2)`	$(1,3,5,4)$	头维提到前面，变成 (B,h,T,d_h)
$qk^\top/\sqrt{d_h}$	$(1,3,5,5)$	每个头一张 $T\times T$ 分数矩阵
softmax	$(1,3,5,5)$	每行归一化
weights $\cdot\, v$	$(1,3,5,4)$	上下文 (B,h,T,d_h)
`transpose` + `contiguous` + `view`	$(1,5,12)$	拼接所有头，回到 d_{model}
`self.proj`	$(1,5,12)$	输出投影

2.5.2 两个实现技巧

第一，**用一个线性层一次算出 QKV**。与其写三个 `Linear`，不如写一个输出维度为 $3d_{\text{model}}$ 的 `Linear`，再在最后一维拆成三份。课件里也强调：从数学上看两者完全等价，但一次大矩阵乘法在 GPU 上更高效。

第二，**`transpose` 之后必须 `contiguous` 才能 `view`**。`transpose` 只改变张量的步长（stride），底层内存并不连续；而 `view` 要求内存连续，否则会报错。所以拼接所有头时，写法分成三步：先 `transpose(1,2)` 把头维换到中间，再 `contiguous()` 让内存连续，最后 `view(B,T,C)` 合并头。讲师顺带对比了 `reshape`：它会返回一份新数据的副本，也能达到目的，但会多一次内存分配。

```
def forward(self, x):                          # (B,T,d_model)
    B, T, C = x.shape
    qkv = self.qkv(x)                          # (B,T,3*C)
    qkv = qkv.view(B, T, 3, self.n_head, self.d_head)
    q, k, v = qkv.unbind(dim=2)                # each (B,T,heads,dim)
    q, k, v = q.transpose(1, 2), k.transpose(1, 2), v.transpose(1, 2)
    scale = 1.0 / math.sqrt(self.d_head)
```

```
    attn = torch.matmul(q, k.transpose(-2, -1)) * scale   # (B,h,T,T)
    mask = causal_mask(T, device=x.device)
    attn = attn.masked_fill(mask, float('-inf'))
    w = F.softmax(attn, dim=-1)
    ctx = torch.matmul(w, v)                                # (B,h,T,d_h)
    out = ctx.transpose(1, 2).contiguous().view(B, T, C)
    return self.proj(out), w
```

Listing 2: 多头自注意力的核心前向（节选自 `part_1/multi_head.py`）

模块初始化处有一个很值得学的「护栏」：`assert d_model % n_head == 0`，否则直接抛异常。如果不整除，d_{head} 会出现小数，后面所有形状都会错；早点报错比在矩阵乘法里看到莫名其妙的形状错误要好得多。另一个工程细节是 `trace_shapes` 开关：打开后每一步都打印张量形状，这正是理解 MHA 最快的方式。

除形状追踪外，课程还提供了可视化脚本：运行 `python orchestrator.py --visualize` 会把每个注意力头的权重矩阵画成热图（图 **??**）。热图的对角线及下方较亮、右上角全黑，正是因果掩码在起作用；同时也能直观看到不同头关注的位置不同——这就是「多头」相对单头的意义：同一个输入被投影到不同的子空间，各自学习一种关系。

2.6 因果掩码：让模型看不见未来

Decoder-only 模型的训练目标是「预测下一个 token」，所以在计算位置 t 的表示时，模型**只能使用位置 $\leq t$ 的信息**，不能偷看后面的 token——否则训练时是在抄答案，推理时根本没有未来信息，模型会崩。

实现非常短：构造一个上三角的布尔矩阵，True 表示「需要被遮住」：

```
import torch

def causal_mask(T: int, device=None):
    """True␣means␣masked␣(disallowed);␣shape␣(1,1,T,T)␣for␣broadcasting."""
    m = torch.triu(torch.ones((T, T), dtype=torch.bool, device=device), diagonal=1)
    return m.view(1, 1, T, T)
```

Listing 3: `part_1/attn_mask.py`：因果掩码

然后在 softmax 之前用 `masked_fill(mask, -inf)` 把被遮住的位置设成负无穷。softmax 对 $-\infty$ 的输出是 0，于是这些位置的注意力权重精确地变成 0，等价于「完全看不见」。为什么用 $-\infty$ 而不是 0？因为若填 0，softmax 之后仍有非零权重；只有 $-\infty$ 才能让概率归零。

讲师用一句 “The cat sat on the mat” 来解释掩码的语义：当模型处理到 “sat” 时，它能看见 “The cat sat”（含自己），但看不见 “on the mat”。把掩码可视化出来就是一张下三角矩阵：第 i 行只有前 i 列是有效的。课程还为掩码写了单元测试（`test_causal_mask.py`），检查「上三角被遮、下三角保留、对角线保留」这三个性质。

最常见的实现错误：掩码方向

`torch.triu(..., diagonal=1)` 遮的是「右上角」（未来位置），保留的是左下角（过去与当前）。如果误写成 `diagonal=0` 或用了错误的方向，模型会连自己当前位置都看不到（或者看到未来），训练损失会以难以察觉的方式变坏。写完掩码后，务必打印出来肉眼确认一次。

2.7 前馈网络：4 倍扩张 + GELU

视频位置说明：FFN 与 block 在视频里的位置

Part 1 的视频在 00:39:30 左右收尾于「位置编码 + 注意力 + 可视化」，而 FFN、残差/LayerNorm 与 Transformer block 的代码讲解，出现在 Part 2 讲者走读 GPT 模型时（约 00:57–01:00）。本讲义按课程 README 的小节划分，把它们作为 Part 1 的收尾内容补全，这样读者能在一处看完整条架构。

注意力负责「在 token 之间混合信息」，前馈网络（FFN）负责「对每个 token 单独做非线性变换」。它把每个位置的 d_{model} 维向量映射到 $4d_{\text{model}}$ 维的隐藏空间，过激活函数，再映射回 d_{model}。这个「先扩张再压缩」的结构像一个小自编码器，给模型更大的容量去拟合复杂函数。

```python
class FeedForward(nn.Module):
    def __init__(self, d_model: int, mult: int = 4, dropout: float = 0.0):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, mult * d_model),
            nn.GELU(),
            nn.Linear(mult * d_model, d_model),
            nn.Dropout(dropout),
        )
    def forward(self, x):
        return self.net(x)
```

Listing 4: `part_1/ffn.py`：带 GELU 的前馈网络

两个设计点值得解释。第一是**扩张倍数 `mult=4`**：GPT 系列常用 4 倍，这是容量与计算量之间的经验折中（Part 3 会看到现代模型如何用门控（SwiGLU）在相近参数量下改进这一块）。第二是**GELU 而不是 ReLU**：ReLU 在 0 处有「硬拐点」，而 GELU 是一条平滑曲线，它对负值不是直接截断而是平滑地压小。讲师的说法是：平滑的激活函数通常带来更稳定的梯度与训练过程，这也是 GPT 系列一直用 GELU 的原因。

2.8 残差连接与 LayerNorm：让深网络能训练

到目前为止每个子层都是「一个会改变表示的函数」。如果直接把很多子层串联起来，深层网络会遇到梯度消失、训练不稳定等经典问题。Transformer block 用两个技巧解决它：

1. **残差连接**（residual / skip connection）：$x \leftarrow x + f(x)$。每个子层只学「增量」，原始信息有一条高速公路直接流过去，梯度可以无衰减地反传；
2. **LayerNorm**：对每个 token 的 d_{model} 维表示做归一化（减均值、除标准差，再乘可学习的缩放与偏移），稳定每层的数值尺度。

具体归一化公式为：

$$\text{LN}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta,$$

其中 x 是某个 token 的 d_{model} 维向量，μ, σ^2 是该向量各维的均值与方差，ϵ 是防止除零的小常数，γ, β 是可学习的缩放与偏移参数。

2.8.1 Pre-norm：先归一化再进子层

把上面两个技巧组合成 block 时，有一个顺序选择：先归一化再进子层（pre-norm），还是先过子层再归一化（post-norm）。课程的实现是 pre-norm：

```python
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_head, dropout=0.0):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadSelfAttention(d_model, n_head, dropout)
        self.ln2 = nn.LayerNorm(d_model)
        self.ffn = FeedForward(d_model, mult=4, dropout=dropout)

    def forward(self, x):
        x = x + self.attn(self.ln1(x))[0]    # 注意力子层 + 残差
        x = x + self.ffn(self.ln2(x))        # FFN 子层 + 残差
        return x
```

Listing 5: `part_1/block.py`：Transformer block（pre-norm + 残差）

注意代码里 `self.attn(...)[0]` 取的是元组第一个元素：MHA 的 `forward` 返回 `(out, weights)` 两项，这里只要输出，不要注意力权重（权重只用于可视化）。pre-norm 的好处是每个子层的输入都被归一化到稳定尺度，深堆叠时训练更稳；代价是最后需要再补一个 LayerNorm（见下节 GPT 的前向）。课程在这里顺带说明：Part 3 会把 LayerNorm 换成 RMSNorm，把「减均值」这一步去掉，这是现代 LLM 的标准做法。

LayerNorm vs BatchNorm

LayerNorm 在**每个 token 自己的特征维**上归一化，因此与 batch 大小、序列长度都无关；BatchNorm 则在 batch 维上统计，遇到变长序列会非常别扭。语言模型几乎只用 LayerNorm（或其变体 RMSNorm），原因就在这里。

2.9 把一切堆起来：Transformer block 与 GPT

2.9.1 从 block 到模型

把 TransformerBlock 重复 N 次，再在最前面接「词嵌入 + 位置编码」、最后面接「LayerNorm + 线性头」，就得到一个可以训练的语言模型。Part 2 的 `model_gpt.py` 正是这么写的（摘要）：

```python
self.tok_emb = nn.Embedding(vocab_size, n_embd)
self.pos_emb = nn.Embedding(block_size, n_embd)
self.drop    = nn.Dropout(dropout)
self.blocks  = nn.ModuleList([Block(n_embd, n_head, dropout)
                              for _ in range(n_layer)])
self.ln_f    = nn.LayerNorm(n_embd)              # 最后的归一化
self.head    = nn.Linear(n_embd, vocab_size, bias=False)

def forward(self, idx, targets=None):
    B, T = idx.shape
    pos = torch.arange(0, T, device=idx.device).unsqueeze(0)
    x = self.tok_emb(idx) + self.pos_emb(pos)     # 词嵌入 + 位置嵌入
```

```
    x = self.drop(x)
    for blk in self.blocks:                    # 堆叠 N 个 block
        x = blk(x)
    x = self.ln_f(x)
    logits = self.head(x)                      # (B,T,vocab_size)
    ...
```

Listing 6: GPT 主干的骨架（据 `part_2/model_gpt.py` 精简）

这个骨架里有两个容易漏掉的点：**最后的 `ln_f`**（pre-norm 结构需要它来归一化最后一层的输出）以及**权重初始化**（线性层与嵌入用均值 0、标准差 0.02 的正态分布，偏置置 0）。讲师提到，初始化虽然简单，但对小模型能否顺利收敛影响很大。

另外，`model_gpt.py` 里的 `CausalSelfAttention` 把 Part 1 手写的「masked_fill + softmax」换成了 PyTorch 的 `F.scaled_dot_product_attention(..., is_causal=True)`：两者语义等价，但后者在 GPU 上会走 flash attention 一类的高效实现，这是工程代码里更常见的写法。

2.9.2 把 Part 1 的零件组装起来

到这里，Part 1 的每个零件都已经实现了。把它们串起来就是一次完整的前向传播：

1. 输入 token ID → 词嵌入（B, T, d_{model}）；
2. 加位置编码（正弦或可学习）；
3. 过 N 个 Transformer block：每个 block 内是「LN → MHA → 残差 → LN → FFN → 残差」；
4. 最后 LN → 线性头 → logits → softmax 得到下一个 token 的概率。

Part 1 只做前向，不训模型；训练所需的损失、数据集、采样留到 Part 2。但讲师强调，理解 Part 1 的形状变化（尤其是 MHA 里 $(B, T, 3d) \to (B, T, 3, h, d_h) \to (B, h, T, d_h)$ 这一串）之后，后面所有部分都会变得可读——因为现代模型只是在同样的骨架上替换组件（RMSNorm、RoPE、SwiGLU、MoE）。

2.10 本章小结

- 现代 LLM 是 decoder-only Transformer：token 化 → 词嵌入 → 位置编码 → 多头注意力（带因果掩码）→ 残差 + LayerNorm → 前馈网络 → 堆叠 N 层 → 线性头。
- 位置编码解决「注意力对顺序不敏感」的问题；正弦编码用不同频率的 sin/cos 生成固定向量，用 `register_buffer` 注册、不参与训练。
- Self-attention 是 $\text{softmax}(QK^\top/\sqrt{d_k})V$；除以 $\sqrt{d_k}$ 是为了防止点积过大让 softmax 饱和、梯度消失。
- 多头注意力把一个 d_{model} 拆成 h 个 d_{head}：一次线性层得到 $3d_{\text{model}}$，view 成 $(3, h, d_h)$ 再拆分，各头并行计算后 `concat`（需要 `contiguous` 才能 `view`）再投影。
- 因果掩码用上三角布尔矩阵 + $-\infty$ `masked_fill` 实现「看不见未来」，保证训练目标与推理行为一致。
- FFN 是「4× 扩张 → GELU → 压缩」；block 用 pre-norm 残差结构（`x + sublayer(LN(x))`）保证深层可训练；堆叠后接最终 LayerNorm 与线性头即为 GPT。

3 Part 2 —训练一个小 LLM

3.1 本讲定位：把 Part 1 的模型真正训起来

Part 1 已经把 Transformer 的每个零件写完了；Part 2 要做的是**把这些零件组装成一条可运行的训练流水线**：从原始文本文件出发，经过字节级分词、批采样、前向与交叉熵、反向与优化器更新，最后能从一个提示词开始自回归地生成文本。

讲者先展示本部分的入口 `orchestrator.py`。它本身不做训练，只负责把参数拼好、依次调用脚本。Part 2 里主要涉及三个文件：

- `train.py`：训练主脚本（分词器、数据集、模型、训练循环、采样都在这里串起来）；
- `tokenizer.py`：字节级分词器 `ByteTokenizer`；
- `dataset.py`：把文本文件变成可采样的 (x, y) 批次。

讲者给出的运行命令（关键参数）如下（图 **??**）：

```
python train.py --data tiny_hi.txt --steps 400 --sample_every 100 \
  --eval_interval 100 --batch_size 32 --block_size 128 \
  --n_layer 2 --n_head 2 --n_embd 128
```

Listing 7: Part 2 的运行命令（据视频 orchestrator 传参整理）。

注意这里的模型极小：**2 层、2 个注意力头、128 维嵌入、上下文窗口 128**——目标是把流程跑通并观察损失下降，而不是训练一个可用的模型。

这一部分刻意“够用就好”

Part 2 的分词器、数据集、训练循环都是**最小可运行版本**：字节分词器词表只有 256、数据集一次性读进内存、模型只有 2 层。讲者反复强调这些实现“不是工业级”，目的是先建立正确的心理模型；真正的工程化（BPE、梯度累积、学习率计划、检查点恢复）放在 Part 4。

3.2 字节级分词器：每个字节就是一个 token

语言模型不直接读字符，而读整数 ID。最朴素的做法是**字节级（byte-level）分词**：把字符串按 UTF-8 编码成一串字节，每个字节（0–255）就是一个 token，因此**词表大小固定为 256**（图 **??**）。

```
class ByteTokenizer:
    """Ultra-simple byte-level tokenizer."""
    @property
    def vocab_size(self) -> int:
        return 256

    def encode(self, s: str) -> torch.Tensor:
        return torch.tensor(list(s.encode("utf-8")), dtype=torch.long)

    def decode(self, ids) -> str:
        if isinstance(ids, torch.Tensor):
            ids = ids.tolist()
        return bytes(ids).decode("utf-8", errors="ignore")
```

Listing 8: 字节级分词器的核心（据视频 `tokenizer.py` 整理）：编码用 UTF-8 字节，解码把字节串还原成文本。

讲者对这个分词器的评价很直接：**词表极小，但序列会变长**——一个汉字在 UTF-8 下通常占 3 个字节，也就是 3 个 token；英文大约 1 字节/token。序列变长意味着同样长度的上下文能装下的“语言内容”变少，注意力计算量也随之上升。所以它适合入门，不适合工业级应用；Part 4 会换成 BPE。

细节坑：`errors='ignore'` 会悄悄吞掉字节

解码时若字节序列不完整（例如模型刚采样到某个多字节字符的前两个字节），`decode(errors='ignore')` 会**丢弃**这段字节而不报错。这在演示里没问题，但工程上要意识到：分词器的 encode/decode 不是严格可逆的，评估生成质量时可能看到“少字”。

3.3 数据集与批采样：随机窗口 + label shift

数据加载的路径非常直接（图 ??）：`Path.read_bytes()` 读入整个文件，转成 `long` 张量；按 90/10 切成训练集与验证集；每次采样时，在 $[0,\ \text{len} - \text{block_size} - 1]$ 里随机取 `batch_size` 个起点，每个起点取长度为 `block_size` 的一段作为输入 x。

关键在于 y：**它不是另做一份标签，而是同一段文本向后平移一位**。第 i 个样本的 y 是 $buf[i+1 : i+1+\text{block_size}]$，于是对于 x 中位置 t 的 token，y 中对应位置就是它的**下一个 token**——这就是语言模型训练所需的 **label shift**。

```python
class ByteDataset:
    def __init__(self, path, block_size=256, split=0.9):
        data = Path(path).read_bytes()
        data = torch.tensor(list(data), dtype=torch.long)
        n = int(len(data) * split)
        self.train, self.val = data[:n], data[n:]
        self.block_size = block_size

    def get_batch(self, which, batch_size, device):
        buf = self.train if which == "train" else self.val
        assert len(buf) > self.block_size + 1
        ix = torch.randint(0, len(buf) - self.block_size - 1, (batch_size,))
        x = torch.stack([buf[i : i + self.block_size] for i in ix])
        y = torch.stack([buf[i + 1 : i + 1 + self.block_size] for i in ix])
        return x.to(device), y.to(device)
```

Listing 9: 批采样与 label shift（据视频 `dataset.py` 整理）：y 是 x 右移一位的同一段文本。

讲者在解释随机起点时特意强调了“为什么上界要减 1”：如果不留缓冲区，当随机到接近文件末尾的位置时，$i + \text{block_size}$ 会越界；同理 y 要额外多取一位，所以上界再减 1。这个细节后面会在“模型输出为什么右移一位”处再次用到。

这个数据集实现不能直接用于大规模数据

讲者明确提醒：`read_bytes()` 会把**整个文件读进内存**，在数 TB / 数万亿 token 的规模上会直接爆内存。工业做法是把数据按 shard 存储成 parquet 一类列式文件，**按 partition 增量加载**，而不是一次性读入。

3.4 模型与交叉熵：位置、残差与损失

模型的整体结构在第一部分已经拆解过，这里讲者只做“串起来”的说明（图 ??）。`GPT` 模块包含：

- **token embedding**：`vocab_size`（本例 256）→ `n_embd`（本例 128）；
- **position embedding**：位置 $0 \dots T-1$ → `n_embd`，与 token embedding 相加；
- **若干个 Transformer Block**：本例 2 个；
- **最终 LayerNorm 与输出头**：`Linear(n_embd, vocab_size)`，输出 logits；
- 权重初始化：线性层与嵌入用均值 0、标准差 0.02 的正态分布，偏置置零（讲者提到这些超参可以自己调，看是否收敛更快）。

前向传播里有两个值得记住的实现选择：

1. **位置编码是“加到嵌入上”的**：`pos = arange(0, T)`，取位置嵌入后与 token 嵌入相加，再进 Block。Part 3 会把它换成 RoPE，届时位置信息改为在注意力内部作用在 Q、K 上。
2. **Block 采用 pre-norm 残差**：先归一化、再进注意力和前馈，然后与原始输入相加。视频里展开的 Block 是 $x \leftarrow x + \mathrm{Attn}(\mathrm{LN}(x))$，$x \leftarrow x + \mathrm{FFN}(\mathrm{LN}(x))$；FFN 是「线性升维 4 倍 → GELU → 线性降回 → dropout」。

当 `targets` 不为空时，模型直接计算交叉熵损失。把 logits 展平成 $(B \cdot T, \text{vocab})$、targets 展平成 $(B \cdot T)$，就是标准的下一 token 分类：

$$\mathcal{L} = -\frac{1}{B \cdot T} \sum_{b=1}^{B} \sum_{t=1}^{T} \log p_\theta\big(y_{b,t} \mid x_{b,1}, \dots, x_{b,t}\big)$$

逐项解释：B 是 batch size；T 是上下文长度（`block_size`）；$y_{b,t}$ 是第 b 条样本第 t 个位置的目标 token（也就是输入右移一位）；p_θ 是模型给该 token 的 softmax 概率。训练时“每个位置都在做一次 next-token prediction”，所以一条长度 128 的样本会贡献 128 个监督信号。

3.5 训练循环：AMP、梯度裁剪、验证与检查点

训练循环是 Part 2 的工程核心（图 ??、图 ??）。讲者按执行顺序拆开讲：

1. **设备与编译**：先用 `torch.cuda.is_available()` 判断是否有 GPU，并且用户没有传 `--cpu`，才把设备设为 CUDA；若开启 `--compile`，用 `torch.compile` 把模型编译成更快的静态图。讲者顺势聊到 PyTorch/TensorFlow 的历史：PyTorch 早年以“动态图、好调试”取胜，TensorFlow 2.0 反过来引入动态执行，两家在趋同；选工具应看任务，不必站队。

2. **优化器**：Adam，学习率、betas、weight decay 都从参数传入。

3. **自动混合精度（AMP）**：用 `torch.amp.autocast` 把部分算子降到 FP16/BF16 计算，用 `GradScaler` 处理**梯度下溢**。

4. **梯度裁剪**：`clip_grad_norm_(params, grad_clip)`，本例 `grad_clip=1.0`，防止个别异常样本产生巨大梯度把模型带偏。

5. **日志、验证与保存**：每 50 步打印 loss；每 `eval_interval` 步在验证集上估损失；验证损失创新低就保存 checkpoint（模型 `state_dict` + 结构超参，便于精确恢复）。

混合精度的因果链值得完整复述一遍：FP16 的指数位少、能表示的最小正数有限；当梯度小于这个范围时会变成 0（**underflow**），对应的参数就“学不动”。GradScaler 的做法是**把 loss 乘以一个缩放因子** S 再反向传播，链式法则会让梯度也放大 S 倍，于是不会下溢；在 `optimizer.step()` 之前再**除以** S（`scaler.unscale_(opt)`），保证实际更新量不变。GradScaler 还会动态调整 S：连续若干步没有溢出就乘 2 增大，一旦溢出就减半（文档里 `init_scale=65536.0, growth_factor=2.0, backoff_factor=0.5, growth_interval=2000`）。

注意 `unscale_` 的位置很关键：**先 unscale、再梯度裁剪、最后 step**。如果先裁剪后 unscale，裁剪阈值作用在放大后的梯度上，等价于阈值被缩小了 S 倍。

```
scaler = torch.amp.GradScaler(enabled=(args.amp and device.type == "cuda"))
for step in range(1, args.steps + 1):
    xb, yb = ds.get_batch("train", args.batch_size, device)
    with torch.amp.autocast(device_type="cuda", enabled=(args.amp and device.type
        == "cuda")):
        _, loss = model(xb, yb)
    opt.zero_grad(set_to_none=True)
    scaler.scale(loss).backward()
    if args.grad_clip > 0:
        scaler.unscale_(opt)
        torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
    scaler.step(opt)
    scaler.update()
```

Listing 10: 带 AMP 与梯度裁剪的训练步（据视频 `train.py` 整理，节选核心顺序）。

验证部分对应 `estimate_loss`：把模型切到 `eval()`，在 `no_grad()` 下从 train/val 各取 `eval_iters` 个批次，前向算 loss 并取平均，再切回 `train()`。这里一个容易忽略的点是：**验证只关心 loss，不关心生成**；生成质量由单独的采样环节观察。

3.6 采样：温度、top-k、top-p 与自回归生成

训练结束后（或在训练中途），脚本会从当前 checkpoint 出发做一次生成（图 ??、图 ??）。生成函数 `generate` 的骨架是标准的自回归循环：

```
@torch.no_grad()
def generate(model, idx, max_new_tokens, temperature=1.0, top_k=None, top_p=None):
    model.eval()
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -model.block_size:]          # 截断到上下文窗口
```

```
        logits, _ = model(idx_cond)                    # (B, T, vocab)
        logits = logits[:, -1, :] / max(temperature, 1e-6)  # 只取最后一个位置
        logits = top_k_top_p_filtering(logits, top_k, top_p)
        probs = torch.softmax(logits, dim=-1)
        next_id = torch.multinomial(probs, num_samples=1)
        idx = torch.cat([idx, next_id], dim=1)
    return idx
```

Listing 11: 自回归生成的核心（据视频 `utils.py` / `model.generate` 整理，简化版）。

为什么只取 `logits[:, -1, :]`？讲者专门用一段动画解释：模型输出与输入长度相同，但**整体右移了一位**——输入 4 个 token，输出 4 个位置的预测，其中最后一个位置才是“基于完整输入预测的下一个 token”。因此生成时每一步只需要最后那一个位置的 logits。

温度（temperature）作用在 softmax 之前：把 logits 除以温度 T。$T < 1$ 时 logits 差异被放大、分布更尖锐（更保守、更确定）；$T > 1$ 时差异被压缩、分布更平坦（更随机、更有创造性）。实现上用 `max(temperature, 1e-6)` 防止除零。

讲者还讨论了 `temperature=0` 的实践：Google 的 Gemini API 文档写的是“温度 0 意味着总是选概率最高的 token，但仍可能有一点点变化”——这说明工业级实现里“温度 0”并不严格等于贪心，因此视频里的代码用 `max(temperature, 1e-6)` 防止除零；而 Part 3 的现代实现则显式地在最低温时改用 `argmax`。

top-k 与 **top-p** 都是在“截断候选集”后再采样：

- **top-k**：保留概率最高的 k 个 token，其余置为 $-\infty$，softmax 后概率为 0、不会被采样到。实现用 `torch.topk` 取第 k 大的值作为阈值，把小于阈值的 logits 置 $-\infty$。
- **top-p（nucleus sampling）**：按概率降序排序、做累积和，只保留“累积概率刚好达到 p”之前的那批 token，长尾全部丢弃。实现上要注意**至少保留一个 token**（把 mask 的第一个位置强制设为不丢弃），否则极端情况下可能一个都不剩。

两种过滤可以叠加；再叠加“排序后过滤、最后用 `torch.scatter` 写回原始顺序”的实现技巧，就是视频里 `top_k_top_p_filtering` 的全部内容。

3.7 实测：损失在降，但模型仍然“不会说话”

讲者最后完整跑了一遍 Part 2（图 **??**）。终端输出里的关键信息有三类：

- **训练损失持续下降**：第 50 步约 2.26，第 100 步约 1.49；之后继续降到 1.34 / 1.28 / 1.24 / 1.23 量级；
- **验证损失同步下降**：例如第 100 步 `train 1.4577 / val 1.4425`，第 200 步 `train 1.2923 / val 1.2769`，说明不是单纯过拟合训练集；
- **生成样本仍然是“乱码”**：输出的是看起来像数据集中语言（本例是印地语文本）的字符片段，但没有语义。讲者解释得很坦率：**2 层、128 维的模型不可能生成有意义的文本**，这条流水线的价值在于“每一步都对、损失确实在降”。

这一讲真正要带走的三件事

1. **分词器决定 token 序列的形状**：字节分词器简单但序列长；工业上换 BPE。
2. **label shift 是语言模型训练的全部“标签来源”**：一条文本自己监督自己，y 就是 x 右移一位。
3. **AMP 的正确顺序是 scale → backward → unscale → clip → step → update**；顺序错了训练会以很难排查的方式变差。

3.8 本章小结

- 训练流水线：`orchestrator` 传参 → `train.py` 串起分词器、数据集、模型与训练循环；本例是 2 层、2 头、128 维、上下文 128 的“玩具模型”。
- 字节级分词器以 UTF-8 字节为 token、词表 256；实现极简但序列会变长，工业场景应换 BPE。
- 数据集直接读字节、按 90/10 切分、随机取窗口；y 由 x 右移一位得到（label shift），这是 next-token prediction 的标签构造方式。
- 交叉熵把每个位置的下一 token 分类损失平均起来；一条长度 T 的样本贡献 T 个监督信号。
- 训练循环的关键工程点是 AMP 的缩放/反缩放顺序、梯度裁剪、周期性验证与保存最优 checkpoint；验证 loss 与训练 loss 同步下降说明训练是健康的。
- 采样由温度控制分布尖锐程度、top-k/top-p 截断候选集；生成时只取最后一个位置的 logits，再自回归拼接。2 层模型能学会字符级统计，但生成不了有意义的文本。

4 Part 3 —架构现代化

4.1 本讲定位：把“2017 年的 Transformer”升级成现代架构

Part 3 不再训练模型，而是**逐个替换架构零件**，把 Part 1/2 里的经典 Transformer 升级成今天常见的现代 LLM 结构。讲者列出的改造清单是：

- **RMSNorm**：替换 LayerNorm；
- **SwiGLU**：替换 FFN 里的 ReLU/GELU；
- **KV Cache**：推理加速（不是架构改动，但决定了后面的设计）；
- **Grouped-Query Attention（GQA）**：在多头与多查询之间折中，压缩 KV Cache；
- **Sliding Window Attention + Attention Sink**：把稠密注意力换成带“注意力汇聚点”的滑动窗口；
- **RoPE（旋转位置编码）**：替换绝对位置嵌入，兼顾相对位置信息与 KV Cache 友好性。

最后的 `demo_generate.py` 用随机权重直接跑生成，**对比有/无 KV Cache 的速度**，从而把“架构改动”和“推理效率”两条线合在一起验证。

4.2 RMSNorm：只除均方根，不减均值

LayerNorm 做的是“减均值、除标准差、再仿射变换”；RMSNorm 把前两步简化成“**只除均方根**”：

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}, \qquad y = \frac{x}{\mathrm{RMS}(x)} \cdot g$$

逐项解释：$x \in \mathbb{R}^d$ 是一个 token 的隐藏向量；d 是隐藏维度；ϵ 是防止除零的极小量；$g \in \mathbb{R}^d$ 是可学习的缩放参数（逐维）。与 LayerNorm 相比，它**不减均值、不加偏置**。

讲者给的理由是**效率**：RMSNorm 只有平方、求均值、开方、除法几次逐元素运算，GPU 上可以更好地融合（fuse）成一个 kernel；而 LayerNorm 还要额外算均值与偏置。代价是表达能力略降，但实践表明这个简化几乎不损效果——这是“用更少的运算换同样的效果”的典型架构优化。

4.3 SwiGLU：用门控 + SiLU 替换 FFN 激活

经典 FFN 是“升维 → 激活 → 降维”。SwiGLU 把它换成三**条线性层 + 门控**（图 **??**）：

$$\mathrm{FFN}(x) = W_3\big(\ \mathrm{SiLU}(W_2 x)\ \odot\ W_1 x\ \big), \qquad \mathrm{SiLU}(z) = z \cdot \sigma(z)$$

逐项解释：W_1 是“值”分支（不做激活）；W_2 是“门”分支，过 **SiLU**（也叫 Swish，即 $z{\cdot}\mathrm{sigmoid}(z)$）；$\odot$ 是逐元素相乘；W_3 把结果投影回隐藏维度。视频里的 `SwiGLU` 类就是 `w1/w2/w3` 三个无偏置线性层加一个 `nn.SiLU()`。

讲者顺带展示了 SiLU 的曲线形状（图 **??**）：在负半轴不是像 ReLU 那样直接归零，而是一个平滑衰减的 S 形，因此比 ReLU 更平滑、梯度更连续——这正是它被现代 LLM 广泛采用的原因之一。

SwiGLU 的代价

门控结构让 FFN 从两条大矩阵变成三条矩阵（`w1/w2/w3`），参数量增加约 50%。因此现代模型通常会把 FFN 的扩展倍数从 4 略微调小，以保持总参数量可比——读实现时不要只看“mult=4”。

4.4 KV Cache：用显存换计算

生成是自回归的：生成第 2 个 token 时，第 1 个 token 的 K、V 与生成第 1 个 token 时完全一样，但朴素实现会把整个前缀重新前向一遍，**重复计算**。KV Cache 的做法是**把每层已经算出的 K、V 存起来**，新 token 只计算自己的 K、V，再与缓存拼接（图 **??**）。

KV Cache 的代价是显存。讲者用白板算了一笔账（图 **??**）：假设一张 FP16 的卡、30B 参数模型、48 层、隐藏维 7168、序列长度约 1024（口述为 1000）、batch size 128，K 和 V 各存一份：

$$2\ (\mathrm{K,V}) \times 2\ \text{bytes} \times 48 \times 7168 \times 1024 \times 128 \approx 1.68 \times 10^{11}\ \text{bytes} \approx 168\ \text{GiB} \approx 170\ \text{GB}$$

逐项解释：第一个 2 表示 K 与 V 各一份；2 bytes 是 FP16；48 是层数；7168 是每层每个 token 的隐藏维；1024 是序列长度；128 是 batch size。结果约 170 GB——**比很多模型的权重大得多**。

KV Cache 只在推理时存在

训练时我们一次前向整条序列、不需要缓存（每个训练样本都是独立的一次并行计算）；KV Cache 是**自回归生成**的加速手段。也正因为它变成了推理侧的主要显存开销，后面才有 GQA、滑动窗口这些“压缩缓存”的架构设计。

4.5 Grouped-Query Attention：在“效果”和“缓存大小”之间折中

三种注意力（图 ??）：

- **MHA（多头注意力）**：每个 query 头都有自己独立的 K、V 头。效果最好，KV Cache 最大。
- **MQA（多查询注意力）**：**所有** query 头共享同一组 K、V。KV Cache 最小、效率最高，但准确率损失明显。
- **GQA（分组查询注意力）**：折中——把 query 头分成若干组，每组共享一组 K、V。组大小（group size）可调。

讲者指出：选对组大小可以**把 KV Cache 降低一个数量级**而效果损失很小，这正是现代模型普遍采用 GQA 的原因。实现上，查询头的数量仍是 `n_head`，K/V 头数量变成 `num_kv_heads`，二者必须整除；前向时用 `repeat_interleave` 把 K、V 复制到与 query 头相同的数量（图 ??）。

4.6 滑动窗口注意力与 Attention Sink

稠密注意力的复杂度是序列长度的平方。当上下文很长时，一个自然的想法是“只看最近的一段”——**滑动窗口注意力（sliding window attention）**。但如果只保留窗口，会出现一个新问题：窗口滑过之后，序列开头的信息彻底丢失，注意力概率被迫摊到窗口内所有 token 上，导致困惑度上升、灾难性遗忘。

Streaming LLM 的解法是保留**前几个 token 作为“注意力汇聚点”（attention sink）**：窗口滑动时，把不想分配的注意力概率“丢”到序列开头那几个固定 token 上（图 ??）。讲者解释这个直觉时强调：汇聚点相当于一个“垃圾桶”，让注意力不必在窗口内平摊；同时窗口内的 token 仍然是最近的上下文。

另一个直觉来自**感受野**：单层滑动窗口只看最近 w 个 token，但多层堆叠后，第 2 层每个位置都“看过”第 1 层 w 个位置的输出，等效感受野随层数增长。讲者用句子“The cat sat on a mat”演示：窗口为 3 时，第 1 层看到 1–3、第 2 层就能覆盖 5 个 token（图 ??）。所以“窗口小”不等于“上下文小”，关键是层数够多。

把“汇聚点 + 滑动窗口”落到 KV Cache 上，就是**滚动缓冲区 KV Cache（rolling buffer KV cache）**：缓存只保留最前面的若干汇聚 token 与最近的窗口 token，中间的直接剔除；实现上就是“取前 `attention_sink` 个 + 取后 `sliding_window` 个，再拼接”。讲者说配合合适的超参，这可以把 KV Cache 再降一个数量级。

4.7 位置编码的进化：绝对 → 相对 → RoPE

Transformer 本身是**位置无关**的：自注意力对输入顺序不敏感，而“猫坐在垫子上”和“垫子上坐着猫”含义完全不同，所以必须注入位置信息。讲者按三段讲这条进化线：

1. **绝对位置嵌入**：给位置 $0, 1, 2, \ldots$ 各学一个向量加到 token 嵌入上。问题是每个位置彼此独立，模型学不到“两个 token 相距多远”这种**相对**关系。

2. **相对位置嵌入**：引入只依赖“距离”的偏置 $B_{-2}, B_{-1}, B_0, B_1, B_2$，距离为 1 的 token 对都用 B_1。它能表达相对距离，但**与 KV Cache 冲突**：新 token 进来时，之前所有缓存项的偏置都要改，无法“缓存后就不动”。

3. **RoPE（旋转位置编码）**：把位置信息编码成**旋转**——第 m 个位置的向量旋转 $m\theta$，任意两个向量的内积就只依赖它们的角度差（相对距离）。每个 token 的旋转只由自己的位置决定，**已经写进 KV Cache 的项不需要再改**。

RoPE 的数学形式在 RoFormer 论文里写得很清楚（图 **??**）：把 d 维空间分成 $d/2$ 个二维子空间，对每一对维度施加旋转矩阵，旋转角与位置 m 成正比，频率为

$$\theta_i = 10000^{-2(i-1)/d}, \qquad i = 1, \ldots, d/2$$

逐项解释：θ_i 是第 i 对维度的旋转频率；10000 是基频（base），由论文设定；d 是头维度；$m\theta_i$ 是位置 m 在该对维度上的旋转角。因为 $R_{\Theta,m}^{\top} R_{\Theta,n} = R_{\Theta,n-m}$，所以 $q_m^{\top} k_n$ 只依赖 $n - m$——这就是“绝对形式的旋转、相对位置的效果”。

实现上不需要真的构造 $d \times d$ 的稀疏旋转矩阵（太浪费），而是**成对取数**：把向量的偶数维与奇数维分成 x_1, x_2，对每一对做二维旋转

$$\begin{pmatrix} x_1' \\ x_2' \end{pmatrix} = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

cos/sin 只依赖位置与频率，可以**预先算好缓存**（称为 RoPE cache，注意与 KV Cache 区分）。讲者特别强调一个实现差异：经典 Transformer 在进入 Block **之前**把位置嵌入加到 token 嵌入上；而 RoPE 是在注意力内部、**分别作用在 Q 和 K 上**（V 不旋转）。

4.8 代码走读：现代 GPT 与带缓存生成

最后讲者带读现代实现（`model_modern.py`、`block_modern.py`、`attn_modern.py`、`rope_custom.py`、`kv_cache.py`）。整体开关被做成配置项：`rms_norm`、`swiglu`、`rope`、`sliding_window`、`attention_sink`、`num_kv_heads`。几个关键点：

- **模型前向多了一个 `kv_cache_list` 与 `start_pos`**：每一层对应一个缓存对象；第一次调用时缓存为 `None`，之后逐步累积。
- **注意力内部**：Q/K/V 分别投影（因为 GQA 下 K/V 的头数少于 Q，不能再像以前那样一次线性层切三份）；对 Q、K 应用 RoPE；把当前 K/V 与缓存 `cat` 拼接；若启用滑动窗口，则做“保留前 `sink` 个 + 后 `window` 个”的裁剪；最后用 `repeat_interleave` 对齐头数。
- **生成时**：第一步把整个 prompt 喂进去（`start_pos=0`），之后每一步只喂**最后一个 token**（`idx[:, -1:]`），`start_pos` 取缓存长度，从而避免重复计算前缀。

实测对比很直观：用随机权重、生成 200 个 token，**有 KV Cache 约 0.07 s，无 KV Cache 约 0.13 s**——接近 2 倍差距；模型未训练，所以输出内容没有意义，这个 demo 只验证“机制正确 + 速度收益”。讲者还写了一系列单元测试（RoPE 形状与数值、GQA 头数、KV Cache 形状），作为“改架构时先保正确性”的实践。

“改架构”的工程顺序

讲者的实际流程值得抄：**先写测试（形状、数值变化、确定性）→ 再逐模块替换（RMSNorm/SwiGLU/RoPE/GQA/窗口）→ 用随机权重跑生成验证不崩 → 最后才谈速度与效果**。架构改动最容易出错的地方是形状与索引，单元测试是最便宜的保险。

4.9 本章小结

- RMSNorm 去掉均值与偏置、只除均方根，运算更少、更易融合；SwiGLU 用“值分支 × SiLU(门分支)”替换普通激活，代价是 FFN 参数变多。
- KV Cache 用显存换计算，是推理侧的主要显存开销；按 30B/48 层/7168 维/序列 1024/ batch 128/FP16 估算约 168 GiB（约 170 GB）。
- GQA 在 MHA 与 MQA 之间折中：多个 query 头共享一组 K/V，可把 KV Cache 降低一个数量级；实现依赖 `repeat_interleave`。
- 滑动窗口注意力只保留近期 token，配合 attention sink（保留序列开头几个 token）避免概率平摊与灾难性遗忘；多层堆叠让等效感受野随层数增长，落成实现即 rolling buffer KV cache。
- 位置编码从绝对到相对再到 RoPE：RoPE 用与位置成正比的旋转让内积只依赖相对距离，同时保持“每个 token 只由自己位置决定”的可缓存性；实现上用成对旋转 + 预计算 cos/sin 缓存（RoPE cache），只作用于 Q、K。
- 现代实现把上述改动都做成配置开关，并用“有/无 KV Cache”的生成计时与单元测试保证正确性；2 倍左右的推理加速来自“不重复计算前缀”。

下一部分预告（视频自述）

Part 4 会把这些现代架构真正用于训练，并补齐 Part 2 跳过的工程项：从字节分词切换到 BPE、**梯度累积**、学习率计划与 warmup、检查点保存/恢复，以及 TensorBoard / Weights & Biases 日志。也就是说，“训练 & 现代化”两块拼起来，才构成一条完整的现代 LLM 训练流水线。

5 Part 4 —扩展规模

5.1 本讲定位：从「跑通训练」到「生产级训练」

前两部分已经把 Transformer 的零件（Part 1）和一条能跑的迷你训练流水线（Part 2）、以及现代架构的若干改进（Part 3）写完了。Part 4 换了一个视角：不再往模型里加新零件，而是**把训练这件事做到生产级**。讲者的原话大意是：模型架构大家讨论得多，但真正盯着训练过程反复调试是很痛苦、也很少被系统讲的——这一部分补的正是这块。

本讲围绕课程仓库 Part 4 里的 `train.py`（以及它调用的 `tokenizer_bpe.py`、`dataset_bpe.py`、`lr_scheduler.py`、`amp_accum.py`、`checkpointing.py`、`logger.py`）展开，覆盖五件事：**分词器从字节级升级到 BPE、梯度累积与混合精度、学习率计划与 warmup、检查点与断点续训、日志与可视化**。讲者在结尾把 Part 3、Part 4、Part 5 合在一起称为「构建生产级 LM 的核心内容」：架构现代化 + 训练稳定性与通信 + MoE 与稠密层的结合。

这一讲不是 scaling law 推导

章节标题叫 *Scaling Up*，但视频的实际内容是**训练工程**：分词器、数值精度、梯度累积、学习率调度、检查点、日志。讲者在本片段中没有展开 scaling law 的公式、也没有讲数据规模的标度实验；分布式/并行的内容在后面 Part 5 结尾被明确说明「不在本课程范围内」(见 Part 5 的图 **??**)。因此本节按视频实际内容组织。

5.2 train.py 的参数面：一次训练需要哪些「旋钮」

讲者从 `train.py` 的入口参数讲起。参数分成三组（图 **??**）：

- **数据与模型**：数据路径、输出目录、是否使用 BPE、词表大小、`block_size`、层数、注意力头数、嵌入维度、dropout；
- **优化与训练**：batch size、epochs、总步数、学习率、warmup 步数、是否使用混合精度（mixed precision）、**梯度累积步数**（`grad_accum_steps`）；
- **工程与运维**：日志间隔、是否用 TensorBoard、每隔多少步保存检查点、保留最近多少个检查点（`keep_last_k`）、设备（CPU/GPU）。

这些旋钮里，后面几节重点讲三个：**梯度累积**（显存换 batch）、**warmup-cosine 调度**（稳定性）、**检查点与恢复**（长训练的生命线）。分词器则是一个容易被忽略、但直接影响序列长度与训练效率的选择。

5.3 分词器升级：从字节级到 BPE

5.3.1 为什么字节级不够

Part 2 的字节级分词器只有 256 个词表项：一个汉字通常在 UTF-8 下占 3 个字节，于是同一个句子会变成很长的 token 序列。序列变长意味着：同样的上下文窗口能装下的内容更少、注意力计算更多、每个 token 携带的语义更少。生产模型需要用更大的词表把常见字符组合「打包」成一个 token——这就是 **BPE（byte pair encoding，字节对编码）**。

5.3.2 BPE 的机制：反复合并最频繁的相邻对

讲者用一个小例子讲清 BPE：从一串字符出发，统计相邻字符对的出现频率，把最频繁的那一对合并成一个新符号。例如最频繁的对是 `AA`，就把 `AA` 记作 `Z`；词表因此从 4 个符号增加到 5 个，但表示同一串字符所需的 token 数减少了。继续迭代：每次选当前最频繁的对合并，直到达到**最小频率阈值**或**词表大小上限**为止。

讲者特别强调两点工程实践：

1. **BPE 训练不是梯度训练**：它只是在语料上收集并合并统计量，没有前向/反向传播；
2. **训练分词器的语料与训练 LM 的语料可以不同**：如果希望分词器理解代码缩进与语法细节，就用代码语料训练它；这样缩进、常见后缀会变成单个 token，而不是拆成许多字节。

本课程用 `tokenizers` 库（Rust 实现，速度快）而不是手写 BPE：一是教学重点在训练而非分词器实现，二是生产环境也建议用成熟实现。初始词表取字节级的 256，再合并到 `vocab_size=8000`；由于设置了最小频率阈值，实际词表可能略小于 8000（讲者展示的 `vocab.json` 里约有 7600 项）。特殊 token 至少要定义 `BOS`（序列开始）、`EOS`（序列结束）与 `PAD`（填充/未知）。

```python
from tokenizers import Tokenizer, models, trainers, pre_tokenizers

tok = Tokenizer(models.BPE())
tok.pre_tokenizer = pre_tokenizers.ByteLevel(add_prefix_space=False)
trainer = trainers.BpeTrainer(
    vocab_size=vocab_size,
    min_frequency=min_frequency,
    special_tokens=["<bos>", "<eos>", "<pad>"],
)
tok.train_from_iterator(iter_texts(data_path), trainer=trainer)
tok.save(tokenizer_dir)          # 产出 vocab.json + merges.txt
```

Listing 12: BPE 分词器的训练与保存（据视频 `tokenizer_bpe.py` 整理）：用 `BpeTrainer` 指定词表大小、最小频率与特殊 token，在文本文件迭代器上训练后保存。

讲者展示的 `merges.txt` 很直观：先是「空格 + 空格 → 双空格」这类合并，往后出现 `-CTION` 这类常见后缀被合并成一个 token。这说明 BPE 学到的不是「单词」，而是**统计上高效的字符组合**——它让 LM 用更少的 token 表示同样的内容。

5.4 数据加载与模型配置的重建

BPE 版本的 `dataset_bpe.py` 与 Part 2 的字节级版本结构相同：把文本读入内存、用 BPE 编码、`get_batch` 随机取窗口并做 label shift（y 是 x 右移一位的同一段文本）。讲者提醒：这里是教学用的小语料所以一次性读入内存，生产环境通常要流式/分片加载。

这里有一个训练脚本里很关键的工程细节：**恢复训练时不能只保存权重**。因为恢复模型需要知道词表大小、层数、头数、嵌入维度等超参数，所以检查点里必须同时保存模型配置（config）与分词器；加载时还要检查「检查点的词表大小」与「当前分词器的词表大小」是否一致，不一致就报错，而不是带着错位的 embedding 继续训练。讲者用「如果没有 config，程序会报错，而不是去猜参数」来说明这一点。

5.5 学习率计划：warmup + 余弦衰减

训练开始就把学习率设到目标值是有风险的：参数是随机初始化的，早期梯度方向不稳定，大学习率容易把训练带偏。Warmup 的做法是**先用小学习率线性升温到基础学习率，再按余弦曲线衰减**（图 **??**）。讲者的解释是：这样可以减少「基础学习率该设多少」的盲目猜测——不必一开始就赌一个正确值，而是让模型先稳定下来。

用公式写出来（t 为当前步，T_w 为 warmup 步数，T 为总步数，η_0 为基础学习率）：

$$\eta(t) = \begin{cases} \eta_0 \cdot \dfrac{t}{T_w}, & t < T_w, \\ \dfrac{\eta_0}{2}\left[1 + \cos\left(\pi \cdot \dfrac{t - T_w}{T - T_w}\right)\right], & t \geq T_w, \end{cases}$$

其中 $\eta(t)$ 是第 t 步使用的学习率，η_0 是基础学习率（`args.lr`，视频示例量级为 3×10^{-4}），T_w 是 warmup 步数（取「用户指定值」与「总步数的 10%」中较小者），T 是总训练步数（取「命令行步数」与「epochs × 数据加载器长度」中较小者）。讲者用「t/T_w 乘以基础学习率」演示了升温过程：前期学习率很小，随 warmup 步数增加而线性变大；之后余弦曲线让学习率平滑下降，保证后期收敛稳定。

5.6 混合精度与梯度累积

5.6.1 混合精度：用更低的数值精度换吞吐

混合精度（mixed precision）的核心是：前向/反向用 fp16 或 bf16 计算，权重与优化器状态仍用 fp32 保存。它带来两个直接收益——显存更省、计算更快（很多 GPU 上低精度矩阵乘吞吐更高）。代价是数值范围变小，梯度可能**下溢**（underflow）成 0。

PyTorch 的解法是 `GradScaler`：反向传播前把 loss 乘一个缩放因子（scale）放大梯度，更新前再除回去；如果检测到 inf/nan 就自动降低缩放因子。讲者的代码里用 `autocast` 包住前向，用 `scaler.scale(...).backward()` 反向，用 `scaler.step(...)` 与 `scaler.update()` 更新。

5.6.2 梯度累积：用显存换「等效大 batch」

梯度累积（gradient accumulation）解决的是「想要大 batch，但显存放不下」的问题。做法是把一个大 batch 拆成 K 个 micro-batch，逐个前向/反向并把梯度累加，等累积满 K 次再真正做一次 `optimizer.step()`。讲者的例子是：想要 batch size 128，但一次前向放不下，就把 `grad_accum_steps` 设为 4，让 4 个 micro-batch 的梯度累加后再统一更新。

为了与「一次大 batch」等价，loss 需要除以累积步数 K：

$$\mathcal{L}_{\text{accum}} = \frac{1}{K}\sum_{k=1}^{K} \mathcal{L}\big(x^{(k)}, y^{(k)}\big),$$

其中 K 是累积步数，$\mathcal{L}(x^{(k)}, y^{(k)})$ 是第 k 个 micro-batch 的交叉熵损失。这样累加出来的梯度就是 K 个 micro-batch 的**平均梯度**，与直接在该大 batch 上算梯度一致。讲者的实现里对应「loss 先除以累积步数，再做 backward」。

讲者把这两个机制封装成一个 `AmpGrad` 类（见 `amp_accum.py`），对外只暴露三个动作：反向、判断是否该更新、执行更新。这样训练循环本身保持干净。下面是根据画面整理的关键接口（内部实现细节以仓库为准）：

```
amp = AmpGrad(optim, accum=args.grad_accum_steps, amp=args.mixed_precision)

with torch.cuda.amp.autocast(enabled=amp.amp):
    logits, loss, _ = model(xb, yb)
amp.backward(loss)                # inside: scaler.scale(loss / accum).backward()

if amp.should_step():
    amp.step(); amp.zero_grad()  # scaler.step(optim); scaler.update(); zero_grad
    lr = sched.step()
step += 1
```

Listing 13: 混合精度 + 梯度累积的关键调用（据视频 `train.py` 画面整理）：autocast 前向、缩放后反向，按累积步数决定何时真正更新。

两个常见坑

(1) 忘了把 loss 除以累积步数，等效学习率会放大 K 倍，训练容易发散；(2) 在累积过程中调用 `optimizer.step()` 或提前清零梯度，会把累加的信号冲掉。判断「是否该 step」的计数器要覆盖所有 micro-batch，而不是按外层 batch 计数。

5.7 检查点、原子保存与断点续训

5.7.1 检查点里要存什么

一次长训练可能跑几小时到几天，中途崩溃是常态。检查点（checkpoint）除了模型权重，还要保存：优化器状态（Adam 的动量与二阶矩）、学习率调度器状态（当前步数）、混合精度 scaler 状态、以及模型配置与分词器信息。少存任何一项，恢复后的训练都不是原来那条曲线。

讲者把保存实现成**原子保存（atomic save）**：先写临时文件，写完后重命名到目标路径，避免「写到一半崩溃」留下损坏的检查点。同时用 `keep_last_k` 做**滚动保留**：每保存一次就删掉过旧的检查点，只留最近 k 个（演示里 $k = 2$，每 50 步保存一次；300 步训练里，150 步的检查点会被 250 步的挤掉，以此类推）。否则磁盘会被大量中间文件撑爆。

5.7.2 SIGINT/SIGTERM：按 Ctrl+C 也要保住当前进度

讲者展示了一个很实用的设计：注册信号处理器，把「收到 Ctrl+C / SIGTERM」变成一个**保存请求标志**，然后在训练循环里检查这个标志，一旦为真就保存当前步的检查点再退出（图 **??**）。这样停止训练不会丢掉最后几十步的进度。

对应到代码（据画面整理）：

```
save_requested = {"flag": False}
def _on_term(sig, frame): save_requested["flag"] = True
signal.signal(signal.SIGTERM, _on_term)
signal.signal(signal.SIGINT,  _on_term)

while step < args.steps:
    for xb, yb in train_loader:
        if step >= args.steps: break
        if save_requested["flag"]:
            atomic_save_all(model, optim, sched, amp, step, out_dir,
                            tok_dir, args.keep_last_k, cfg_build)
            print(f"[signal] Saved checkpoint at step {step} to {out_dir}. Exiting.
                ")
            return
        ...
```

Listing 14: 信号处理与「保存当前步」：把 Ctrl+C 变成一次保存请求，而不是直接杀掉进程。

讲者现场演示了效果：训练到第 180 步时按 Ctrl+C，日志显示它在 180 步保存并退出；重新运行时提示「检测到已训练 180 步」，从 180 步继续，而不是退回上一个检查点（150 步）。**多保住 30 步**听起来不多，但在长训练里就是真金白银的算力。

5.8 日志与可观测性：训练到底健康不健康

训练不是「loss 降了就行」。讲者用 TensorBoard 记录了一组指标，构成一个最小的训练观测面板（图 **??**）：

- **loss 与学习率**：loss 在 warmup 后沿余弦计划平稳下降；
- **吞吐**：每秒处理的 token 数（tokens/sec），用来判断训练是否被数据/通信拖慢；
- **GPU 显存**：显存分配量，判断是否接近 OOM；

- **梯度与参数的整体 L2 范数**：讲者观察到两者随训练**逐渐减小**，意味着更新幅度收敛、训练更稳定；
- **文本采样**：每隔若干步真正生成一段文本，人工看质量是否随训练变好；
- **QKV 直方图**：观察注意力投影的数值分布是否稳定、方差是否收敛。

讲者还强调一个细节：**loss 曲线不能替代真实评测**。训练时模型只在记录过的数据分布上前进，部署/生成时历史由模型自己产生，会遇到训练没见过的状态；所以除了 loss，还要定期采样生成、看质量变化。

5.9 本章小结

1. Part 4 名为 *Scaling Up*，实际讲的是**生产级训练工程**：分词器、混合精度、梯度累积、学习率调度、检查点恢复、日志观测。
2. 分词器从字节级升级到 BPE：BPE 通过反复合并最频繁的相邻对扩大词表（256 → 8000）、缩短序列；分词器语料可与 LM 训练语料不同，代码/多语言场景要专门覆盖。
3. 混合精度用低精度前向/反向换吞吐与显存，用 `GradScaler` 防梯度下溢；梯度累积用 micro-batch 累加凑出等效大 batch，loss 必须除以累积步数。
4. 学习率用 warmup + 余弦：先线性升温稳定训练，再平滑衰减；warmup 减少了对基础学习率的盲目猜测。
5. 检查点要保存模型/优化器/调度器/scaler/配置；用原子保存防损坏、用 `keep_last_k` 滚动保留、用信号处理在 Ctrl+C 时保存当前步并精确断点续训。
6. 可观测性是一等公民：loss/lr、吞吐、显存、梯度与参数的 L2 范数、QKV 直方图、样本文本采样，共同构成「训练是否健康」的判断依据。

6 Part 5 —Mixture-of-Experts

6.1 本讲定位：把 FFN 也「现代化」

Part 3 现代化了注意力块（GQA、RoPE 等）；这一部分转向 Transformer block 内部的另一大块——**前馈网络（FFN / MLP）**。讲者的推进方式是「拆小、放大、只研究一个模块」：不训练完整 LM，而是把 MLP 部分单独拿出来，用一个小 demo（6 个 token、4 个专家、top-k=1）把混合专家（Mixture-of-Experts, MoE）的路由、门控与负载均衡跑通并打印出中间量。

6.2 核心思想：稀疏激活换参数容量

MoE 的结构可以这样理解：注意力块的输出不再直接送进一个 MLP，而是先经过一个**门控函数 / 路由器（router）**，它输出一个概率分布，决定把这个 token 送到哪些专家（expert）去处理；每个专家本身就是一个 MLP。被选中的专家处理完再组合起来继续往前传。

这么做的收益是双重的：

- **参数更多**：专家数 E 个，模型总参数量随之增加，容量更大；

- **计算更省**：每个 token 只激活 top-k 个专家，而不是把全部 E 个专家都算一遍——稀疏激活意味着**同样的推理算力下可以有更多参数**，吞吐更高。

讲者的直觉解释是：可以让一个专家擅长编程、另一个擅长语言……但**哪个专家处理哪类 token 不是人指定的，而是训练过程自己学出来的**。多层堆叠后，模型「看起来参数很多」，但推理时只激活一小部分。

MoE 的参数量与激活量

若单个专家参数量为 P_e、专家数为 E、每 token 激活 k 个专家，则每个 token 实际参与的专家参数约为 $k \cdot P_e$，而模型**总**专家参数为 $E \cdot P_e$。MoE 的卖点正是「总参数 $E \cdot P_e$ 很大、单 token 计算量只有 $k \cdot P_e$」——用存储换计算效率。若与稠密 FFN 混合，还要加上稠密分支的参数与计算。

6.3 参考实现：Switch Transformer 的简化版

实现遵循 Switch Transformer 这篇早期 MoE 工作（图 ??、图 ??）。论文的目录本身就说明了 MoE 的几个关键主题：稀疏路由（simplifying / efficient sparse routing）、scaling 性质、以及数据/模型/专家并行。本讲只取其中**路由、门控与负载均衡**的最小实现；论文里的并行训练部分不在本课程实现范围内（见本节末尾图 ??）。

6.4 组件一：专家 MLP

每个专家就是一个普通的 MLP，结构与 Part 3 的 FFN 相同：`Linear` 升维、`SiLU/SwiGLU` 激活、`Linear` 降维，可选 dropout。讲者的配置是「乘数 4」：输入维度 128 时，隐藏层为 $128 \times 4 = 512$。四个专家就是四个这样的 MLP；它们结构相同、参数独立。

6.5 组件二：Top-K 门控（gating）

门控函数本身非常轻量：一个从 d_{model} 到专家数 E 的线性层，输出 logits，softmax 成概率，再用 `torch.topk` 取前 k 个专家。讲者的实现把「路由」与「负载均衡的统计量」放在同一个 `TopKGate` 里（图 ??）。

关键代码（据画面整理，保留原变量名）：

```
class TopKGate(nn.Module):
    def __init__(self, dim, n_expert, k=1):
        super().__init__()
        assert k >= 1 and k <= n_expert
        self.n_expert, self.k = n_expert, k
        self.w_g = nn.Linear(dim, n_expert, bias=True)

    def forward(self, x):                          # x: (S, C)
        logits = self.w_g(x)                       # (S, E)
        probs = torch.softmax(logits, dim=-1)      # (S, E)
        topk_vals, topk_idx = torch.topk(probs, k=self.k, dim=-1)
        S, E = probs.size(0), probs.size(1)
        importance = probs.mean(dim=0)             # (E,) 每个专家的平均被选概率
        hard1 = topk_idx[:, 0]                     # (S,) top-1 硬分配
        load = torch.zeros(E, device=x.device)
```

```
        load.scatter_add_(0, hard1, torch.ones_like(hard1, dtype=load.dtype))
        load = load / max(S, 1)                    # (E,) 每个专家分到的 token 比例
        aux_loss = (E * (importance * load).sum())
        return topk_idx, topk_vals, aux_loss
```

Listing 15: Top-K 门控与负载均衡辅助损失（据视频 `gating.py` 整理）：概率矩阵按 token 求专家维均值得到 importance，按 top-1 硬分配统计 load，二者相乘求和再乘专家数得到 aux loss。

6.6 组件三：MoE 层的前向（token 展平与按专家填回）

MoE 层前向的工程要点是**维度的展平与还原**。输入形状是 (B, T, C)（batch、序列长度、通道），但路由是按 token 独立决策的，所以先把 $B \times T$ 压成一维 token 流，得到 (S, C)（$S = B \cdot T$），送进门控；再按门控给出的索引，把每个 token 分给对应专家处理；最后把结果 reshape 回 (B, T, C)。

讲者特意把 6 个 token 的输入**拆成两个 micro-batch** 再拼接，模拟真实训练中「一个 batch 的序列在展平后如何合并成一条 token 流」：展平后每一行是一个 token，每一列是一个专家的概率；按 top-k 选出专家后，用 scatter 类操作把激活填回对应专家的输出张量。

6.7 负载均衡：为什么需要辅助损失

6.7.1 问题：专家「名人化」

如果放任路由器自由学习，会出现讲者所说的「名人问题」（celebrity problem）：大多数 token 都挤到少数几个专家，其他专家几乎不被使用、也学不到东西。这既浪费参数容量，也让训练不稳定（专家的梯度分布极不均衡）。所以需要显式的**负载均衡损失**（load-balancing loss）来惩罚不平衡的路由。

6.7.2 机制：importance × load 的辅助损失

Switch Transformer 的辅助损失由两个统计量构成：

$$\mathcal{L}_{\text{aux}} = E \cdot \sum_{i=1}^{E} f_i \cdot P_i,$$

其中 E 是专家数，P_i 是专家 i 的 **importance**——在该 batch 的所有 token 上，门控给专家 i 的平均概率；f_i 是专家 i 的 **load**——被 **top-1 硬分配**到专家 i 的 token 比例。直观理解：P_i 反映路由器「想让」专家 i 处理多少，f_i 反映它「实际」处理了多少；两者越不平衡，乘积项越倾向于惩罚集中。$\mathcal{L}_{\text{aux}}$ 与语言模型的交叉熵损失相加后一起反向传播。

这个 demo 的数字很好地把机制讲清楚：4 个专家的 load 计数是 $[1, 2, 2, 1]$，说明路由没有塌到单一专家；importance 四者接近（约 0.24–0.27），说明概率分布相对分散；由此算出的 $\mathcal{L}_{\text{aux}} \approx 1.0034$，是一个「接近平衡」的示例。讲者还手算了一个更不平衡的例子（$\mathcal{L}_{\text{aux}} \approx 1.3034$）来展示：路由越集中，辅助损失越大。

辅助损失不是越小越好

$\mathcal{L}_{\text{aux}}$ 的作用是**约束路由别塌缩**，而不是训练目标本身。训练时它按一个系数加到总损失上；系数过大可能牺牲主任务效果（强迫均匀路由），过小则起不到均衡作用。本讲演示了 loss 的计算与量级，但没有展开系数调参——实际训练中它通常需要与主损失一起权衡。

6.8 与稠密层混合：hybrid 架构

讲者最后介绍了「MoE + 稠密 FFN」的混合做法：把 MoE 的输出与一个普通稠密 FFN 的输出加权融合，

$$y = \alpha \cdot \mathrm{MoE}(x) + (1-\alpha) \cdot \mathrm{DenseFFN}(x),$$

其中 $\alpha \in [0,1]$ 控制两者比重，视频里的默认值取 0.5（等权）。这样可以在「保留稠密层稳定行为」与「引入专家容量」之间平滑过渡：从纯稠密（$\alpha = 0$）到纯 MoE（$\alpha = 1$）可以渐变，而不是一刀切替换。讲者提到这种混合是「把 MoE 与稠密层结合以构建混合架构」的一种简单方式。

6.9 范围说明：分布式与训练-推理差异不在本讲

Part 5 的 README 提纲里列了「训练稳定性与分布式环境下的通信开销」(5.3)，但视频里讲者明确说明：**分布式部分不在本课程范围内**（图 **??**），他原本计划讲、后来决定不讲。因此本讲实现的是**单机 MoE 模块**：路由、门控、辅助损失、与前向的 token 展平/还原。

视频未覆盖、但面试常问的三个 MoE 概念（补充背景，非本讲内容）

(1) **capacity factor**：每个专家能接收的 token 上限 = 容量因子 × 平均分配量；超出容量的 token 会被丢弃或走残差（token dropping）——这是 Switch Transformer 工程实现里的重要机制，本讲代码没有实现。(2) **专家并行（expert parallelism）**：专家分布在不同设备上，token 需要 all-to-all 通信才能在对应设备上计算，这是 MoE 训练/推理的主要通信开销来源。(3) **训练与推理的差异**：训练时一个 batch 内 token 多、路由统计稳定；推理时 batch 小、负载波动大，工程上常需要 capacity 上限与路由缓存等优化。这些都不在本讲代码范围内，回答面试时要区分「本讲实现」与「原论文/工程实现」。

6.10 本章小结

1. MoE 把 Transformer 的 FFN 换成「路由器 + 多个专家 MLP」：总参数随专家数增加，但每个 token 只激活 top-k 个专家，用稀疏激活换容量与吞吐。
2. 门控就是一个 `Linear(dim, n_expert)` + softmax + `topk`；专家是结构相同、参数独立的 MLP（本讲用「乘数 4」的隐藏层）。
3. MoE 前向的关键工程是把 (B, T, C) 展平成 token 流 (S, C) 再还原；路由按 token 独立决策，按索引把激活填回对应专家。
4. 负载均衡靠辅助损失 $\mathcal{L}_{\text{aux}} = E \sum_i f_i P_i$：$P_i$ 是平均路由概率（importance），f_i 是被硬分配到专家的 token 比例（load）；它抑制「名人问题」，防止路由塌缩到少数专家。
5. demo 的 6 token / 4 专家例子给出具体数字：probability 矩阵、importance 约 0.24–0.27、load 计数 $[1, 2, 2, 1]$、$\mathcal{L}_{\text{aux}} \approx 1.0034$；越不平衡辅助损失越大（手算例约 1.3034）。
6. 可以用 α 把 MoE 与稠密 FFN 混合（$y = \alpha\,\mathrm{MoE} + (1-\alpha)\,\mathrm{DenseFFN}$），在本讲默认等权；分布式训练与 capacity factor 不在本讲范围内。

7 Part 6 —监督微调（SFT）

预训练让模型学会了「预测下一个 token」，但一个只会续写互联网文本的基座模型并不会自动变成助手：它不知道什么时候该回答问题、什么时候该拒绝、回答应该用什么格式。监督微调（Supervised Fine-Tuning，SFT）就是把这台「文本续写机」调成「指令助手」的阶段——用「指令 + 理想回答」的示范数据继续训练基座模型。把它放进整条流水线看：Part 2–5 训练出基座模型，Part 6 做 SFT，Part 7 训练奖励模型，Part 8 再用 PPO 做 RLHF。本节沿着代码实现走一遍 SFT 的四个关键问题：数据怎么来、对话模板怎么定、损失只算回答部分、以及如何采样与评测。

SFT 在流水线中的位置

GPT 助手式训练分四阶段：预训练（万亿 token 互联网文本，几千张 GPU、数月）→ SFT（约 1 万–10 万条专家示范，1–100 张 GPU、数天）→ 奖励建模（10 万–100 万对偏好比较）→ 强化学习（约 1 万–10 万条 prompt，用 RM 打分做 PPO）。SFT 的算力比预训练小几个数量级，但它决定了模型「像不像一个助手」。

7.1 6.1 工程入口：从基座 checkpoint 出发

训练脚本 `train_sft.py` 的结构与前几个阶段一致：`orchestrator.py` 负责调度，`train_sft.py` 负责训练，`sample_sft.py` 负责采样评测。命令行参数直接体现 SFT 的关键配置：

```
p.add_argument('--data', type=str, default='huggingface')
p.add_argument('--ckpt', type=str, required=False)      # 基座模型 ckpt
p.add_argument('--out', type=str, default='runs/sft')
p.add_argument('--steps', type=int, default=200)
p.add_argument('--batch_size', type=int, default=8)
p.add_argument('--block_size', type=int, default=256)
p.add_argument('--n_layer', type=int, default=4)
p.add_argument('--n_head', type=int, default=4)
p.add_argument('--n_embd', type=int, default=256)
p.add_argument('--lr', type=float, default=3e-4)
p.add_argument('--bpe_dir', type=str, default='../part_4/runs/part4-demo/tokenizer'
    )
```

Listing 16: train_sft.py 的主要命令行参数（按视频画面整理）

和预训练相比，SFT 的模型结构不变，只是数据与损失变了；真正要改的是「输入怎么格式化」和「哪些 token 参与损失」。训练从 Part 5 保存的基座 checkpoint 继续，而不是从随机权重开始。

7.2 6.2 指令数据：从 instruction/input/output 到 (prompt, response)

数据侧用 `LazyTinyHF` 封装，核心数据类 `SFTExample` 只有两个字段：`prompt` 与 `response`。数据来自公开指令数据集 `tatsu-lab/alpaca`（约 5.2 万行），每行有四列：

- `instruction`：要求助手执行的任务；
- `input`：任务需要的输入（可选，例如「反转链表」的代码或列表）；
- `output`：期望助手给出的回答；
- `text`：把 instruction、input、output 拼好的完整文本（部分数据直接给现成格式）。

构造训练样本时，**prompt = instruction + input**，**response = output**；如果数据集不可用（例如断网），则回退到内置的极小样本集，保证流程能跑通。

为什么先用极小数据「过拟合」

讲者反复强调：先用 3 条样本训练，观察损失是否下降、输出是否符合预期，再扩大数据规模。这不是为了得到好模型，而是为了验证「数据管线 + 模板 + 损失 + 采样」整条链路没有 bug——小数据过拟合是最便宜的端到端测试。

此外还有一个 `LengthCurriculum` 封装：给定索引 i 返回第 i 个样本，配合采样器从数据集中反复取样本组 batch。名字叫「课程」，本质是一个可按索引取样的迭代器，方便后面扩展成长度分桶或课程学习。

7.3 6.3 对话模板：把 (prompt, response) 变成一段文本

模型看到的不是结构化字段，而是一段按固定模板拼好的字符串。本讲的模板是：

```
'<s>\n###␣Instruction:\nFirst␣prime␣number\n\n###␣Response:\n'
```

Listing 17: SFT 对话模板（渲染后的实际字符串）

其中 `<s>` 是序列开始 token（SOS，词表 ID 0），回答之后还会追加序列结束 token（EOS，ID 1）。两类格式化函数必须区分：

- `format_example(instruction, response)`：训练用，拼出「SOS + Instruction + Response: + 回答 + EOS」的完整序列；
- `format_prompt_only(instruction)`：推理用，只拼到「Response:」为止，并把模板里可能带出的 EOS 去掉。

为什么要单独写第二个函数？训练时我们需要「输入 + 标准答案」来算损失；推理时模型只拿到用户的 prompt，答案要由它自己生成。如果把 EOS 也喂给模型当输入，模型可能从 EOS 之后继续「续写」，输出乱码——所以推理前要显式去掉 EOS。

7.4 6.4 批处理与 loss mask：只让回答参与损失

这是 SFT 实现中最容易写错、也最重要的一步。Collator（`SFTCollator`）拿到一个 batch 的 (prompt, response) 后做四件事：

1. 用 BPE tokenizer 分别对「格式化后的 prompt」和「完整样本」编码，得到 `prompt_ids` 与 `full_ids`；
2. 令输入 x 为完整序列的 token；令标签 y 为 x 的**右移一位**（$y_t = x_{t+1}$），这样每个位置预测的是下一个 token；
3. 把 y 中属于 prompt 的部分全部替换为 `-100`，末尾最后一位也置为 `-100`（它没有下一个 token 可预测）；
4. 把 x 用填充 token（ID 2）补齐到 `block_size`，y 对应位置补 `-100`。

-100 是 PyTorch 交叉熵的默认忽略索引 `ignore_index`：损失函数遇到它直接跳过，不产生梯度。于是 loss 只在 response 的 token 上计算——这正是我们想要的「只学怎么回答，不用学怎么复述问题」。

设第 i 条样本的序列长度为 T，response 有效位置集合为 $\mathcal{R}$（即 $y_t \neq -100$ 的位置），则 SFT 损失为

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|\mathcal{R}|} \sum_{t \in \mathcal{R}} \log p_\theta(y_t \mid x_{<t}),$$

其中 $p_\theta(y_t \mid x_{<t})$ 是模型在位置 t 对「下一个 token」预测为 y_t 的概率，$|\mathcal{R}|$ 是回答 token 数。直观上：把问题遮住，只对答案的每个 token 做最大似然。

```python
# 1) 只格式化 prompt，得到 prompt 的长度
prompt_ids = tokenizer.encode(format_prompt_only(instruction))
full_ids   = tokenizer.encode(format_example(instruction, response))
prompt_len = len(prompt_ids)

# 2) y 是 x 的右移一位：x[t] -> y[t+1]
x = full_ids
y = x[1:] + [IGNORE]          # IGNORE = -100

# 3) 把 prompt 对应的标签全部遮成 -100
for t in range(prompt_len):
    y[t] = -100

# 4) 填充到 block_size：x 补 PAD(2)，y 补 -100
x = x[:block_size] + [PAD] * (block_size - len(x))
y = y[:block_size] + [-100] * (block_size - len(y))
```

Listing 18: collate 中的核心逻辑（按视频解释重写，细节以仓库代码为准）

这一阶段最常见的三类 bug

（1）忘记右移标签，导致模型被训练成「复制当前 token」；（2）mask 的边界算错一位——prompt 长度和右移后的位置必须对齐，否则问题 token 会漏进损失；（3）填充位置没有置为 -100，让模型花容量去预测「填充 token」。视频里讲者在写 collate 测试代码时就踩过这些坑。

7.5 6.5 训练循环：前向、反向与工程细节

有了 batch 的 (x, y)，训练就是标准循环：前向得到 logits，交叉熵（`ignore_index=-100`）得到 loss，反向传播更新参数。讲者提到两点工程细节：这里可以开启混合精度训练（AMP），也可以做梯度累积以在有限显存下等效放大 batch；训练中每 20 步打印一次 loss，训练结束后用 `torch.save` 保存 SFT 模型。为了便于调试，整个项目配置了 VS Code 的 launch 参数，可以直接在 collator/trainer 里打断点，检查中间张量的形状与数值。

7.6 6.6 采样评测：提前停止与 KV cache 复用

训练完成后用 `sample_sft.py` 采样：加载 SFT checkpoint、设置 block size/层数/头数/嵌入维度、最大生成 token 数、温度与 top-k；把用户 prompt 用 `format_prompt_only` 格式化后编码，调用模型的 `generate`，逐 token 生成直到遇到 EOS（提前停止）或达到最大长度，最后解码打印。

调试 `generate` 时可以看到几个关键现象：对 logits 做 top-k/top-p 过滤后，绝大多数候选被置为 $-\infty$，概率集中到少数 token 上；第一次前向不使用 KV cache，之后每一步只输入最后一个 token、其余历史来自 KV cache（步骤 2 的 `start_pos`=31 表示已有 31 个 token 的缓存）；一旦采样到 EOS（ID 1）就跳出循环。

SFT 阶段可以改变上下文窗口

SFT 可以设置与基座模型不同的 `block_size`。原因是注意力与位置编码的矩阵形状是通用的，同一套权重可以在不同上下文长度下继续训练。工程意义很大：基座用较小窗口（如 8K）训练以省算力，SFT 数据少（千条量级）时把窗口扩到 64K，就以很低的成本获得了长上下文能力。视频在本段专门补充说明了这一点。

7.7 6.7 从玩具数据到规模化：过拟合与泛化边界

先在 3 条样本上训练：损失下降、训练集内问题能答对，但样本外问题明显退化（问 DNA 生成 compass、让写程序却重复训练片段）。然后把 `sample_dataset` 设为 False，从 Alpaca 加载 24 条数据重新训练：初始 loss 很高，随后健康下降；模型能回答训练分布内的问题（如三原色、DNA 的全称），也能输出一段代码。但请注意：这只是一个 2 层 Transformer 的玩具模型，讲者明确说它不会聪明——目的是演示生产级 SFT 的流程，而不是复现一个可用的助手。真实场景中通常是拿别人预训练好的基座（甚至指令微调过的模型）再用大量数据微调；LoRA 等参数高效微调因为网上资源丰富，在本系列中跳过（本课程坚持「从零实现每个模块」）。

关于灾难性遗忘的说明

视频这一部分用「3 条样本过拟合、样本外退化」演示了小数据 SFT 的泛化边界，但没有单独展开灾难性遗忘（catastrophic forgetting）的系统分析。工程上可以把这里的现象当作预警：小数据、单分布、少步数的 SFT 容易让模型只记住样本；真实训练中需要监控基座模型在通用能力上的回退（例如保留一部分通用数据、控制学习率与步数、做训练前后评测）。这部分属于基于本讲实验的延伸，不是视频原文结论。

7.8 本章小结

1. SFT 是在基座 checkpoint 上用「指令 + 回答」示范数据继续训练，把文本续写模型调成助手；算力远小于预训练，但决定交互行为。
2. 指令数据经 `instruction/input/output` 构造为 (prompt, response)，prompt = instruction + input，response = output；先用极小数据过拟合验证管线，再扩大规模。
3. 模板决定模型看到的文本格式。训练阶段用 `format_example`，它包含回答与 EOS；推理阶段用 `format_prompt_only`，它不含回答、并去掉 EOS。
4. loss mask 是核心：y 右移一位、prompt 与填充位置置 `-100`，交叉熵只对回答 token 计损失。

5. 采样时用 top-k/top-p 过滤、KV cache 复用与 EOS 提前停止；SFT 阶段还可以改变上下文窗口，低成本扩展长上下文能力。

6. 小数据只用于验证管线；真实 SFT 依赖更大规模数据与更强基座，且要关注过拟合与基座能力回退风险。

8 Part 7 —奖励建模

SFT 让模型「会回答问题」，但它还不会「按人类偏好回答」：同一问题可以有很多种说法，有的更中立、有的更符合某种角色、有的更安全。奖励建模（Reward Modeling）的目标是训练一个能对「(指令, 回答)」打分的模型，把人类偏好变成一个可优化的标量信号，供下一阶段的强化学习使用。本节先看它在流水线中的位置，再依次讲偏好数据、编码器式奖励模型、masked mean pooling、Bradley-Terry 损失与训练评测。

8.1 7.1 为什么需要奖励模型

RLHF 分两步：先训练奖励模型（本部分），再用它作为奖励信号做强化学习（Part 8 的 PPO）。奖励模型的输入是「指令 prompt + 一条回答」，输出一个标量分数 $r(x, y)$；训练数据不是「标准答案」，而是**偏好比较**：同一个 prompt 下两条回答，人类标注者选出更好的那条。这个阶段的数据成本更高——通常要雇佣承包商甚至领域专家（有高级学位）来保证标注质量，规模也比 SFT 数据大一些。

偏好可以来自很多维度：更中立的语气、更符合设定的角色、避免政治偏见、更安全等。本质上，奖励模型学的是「人类比较两个回答时会选哪个」这一判断。

8.2 7.2 偏好数据的结构与加载

本讲使用 Anthropic 的偏好数据集 `Anthropic/hh-rlhf`：训练集约 16.1 万行，分 train/test 两个 split，每行只有两列——`chosen`（被选中的回答）与 `rejected`（被拒绝的回答）。每条样本的文本里包含完整上下文：指令、prompt，有时还有多轮对话历史，然后才是这一轮的回答。加载后把每行组织成三元组 `(prompt, task, response)`：由于 hh-rlhf 没有独立的 prompt 字段，讲者把 prompt 留空占位，instruction 与对话历史都拼在 chosen/rejected 文本中；如果换用带 prompt 字段的数据集，这个占位符可以直接复用。

数据集差异会传导到 collator

hh-rlhf 没有独立的 prompt 字段，所以 pair collator 用空字符串占位；换数据集时如果它有 `prompt` 字段，把占位符替换成真实 prompt 即可。不要把「空 prompt」当成 bug——它是为兼容不同偏好数据集保留的基础框架。

8.3 7.3 奖励模型架构：encoder-only Transformer + scalar head

实现上采用**仅编码器（encoder-only）**的 Transformer，而不是复用解码器 LM，原因在注意力掩码：

- 解码器（decoder）是因果掩码，第 t 个 token 只能看到前 t 个 token；

- 编码器（encoder）没有因果掩码，每个位置都能看到整段文本——而给「指令 + 回答」整体打分时，我们希望模型能同时看到回答的开头和结尾，因此 encoder 更自然。

架构上，先把 token embedding 与 position embedding 相加，再堆叠 N 层编码器层。每层直接使用 PyTorch 自带的 `torch.nn.TransformerEncoderLayer`（按「Attention is all you need」实现），之后做层归一化，最后接一个线性头，把 d 维表示映射成 1 个标量。与语言模型相比，奖励模型可以小几个数量级——它不做逐 token 生成，只需要给整段文本一个偏好分数。当然也可以走另一条路：拿现成的解码器 LM，在层之后切掉 LM head，换成一个输出标量的线性头；本讲为了展示更多样的实现，选择了 encoder 路线。

两种奖励模型实现怎么选

复用解码器 LM + 线性头：能直接继承已训练的语言能力，适合大规模场景，但需要处理因果掩码（只能看左侧）与显存。encoder-only：结构简单、双向可见全文、参数少，适合从零教学与轻量场景。两条路线在工业界都有使用，本讲选后者主要是为了「看到不同实现」。

8.4 7.4 前向：padding mask 与 masked mean pooling

设 batch 大小为 B、上下文长度为 T、隐藏维度为 d。前向过程与形状如下：

1. 输入 token ID 形状 $B \times T$；token embedding 与 position embedding 相加，得到 $B \times T \times d$；
2. 由输入构造 padding mask（判断每个位置是否为填充 token），形状 $B \times T$。这个掩码会传给 encoder 的 `src_key_padding_mask` 参数，让注意力忽略填充位置；
3. 经过 N 层 encoder 与层归一化，形状仍为 $B \times T \times d$；
4. **masked mean pooling**：把 padding mask 扩展成 $B \times T \times 1$ 的浮点掩码，与表示逐元素相乘（填充位置的 d 维向量全变 0），沿序列维求和得到 $B \times d$，再除以非填充 token 数（mask 求和）得到平均表示；
5. 线性头把 $B \times d$ 映射成 $B \times 1$ 的偏好分数 r。

设 $H \in \mathbb{R}^{B \times T \times d}$ 为 encoder 输出，$m \in \{0,1\}^{B \times T}$ 为「非填充」掩码，则第 b 条样本的池化表示为

$$h_b = \frac{\sum_{t=1}^{T} m_{b,t}\, H_{b,t}}{\sum_{t=1}^{T} m_{b,t}}, \qquad r_b = w^{\top} h_b + c,$$

其中 $H_{b,t} \in \mathbb{R}^d$ 是第 t 个 token 的表示，$m_{b,t} = 1$ 表示该位置不是填充，分母保证只对真实 token 取平均，$w \in \mathbb{R}^d$、c 是线性头参数，r_b 即该样本的标量分数。

三个必须记住的形状

（1）encoder 输入/输出：$B \times T \times d$（本例 $8 \times 256 \times 128$）；（2）padding mask：$B \times T$，扩展后 $B \times T \times 1$；（3）池化后：$B \times d$，再过线性头变成 $B \times 1$。形状对不上时，优先检查 mask 的维度与广播。

8.5 7.5 Bradley-Terry 损失与边际排序损失

偏好建模的经典假设是 Bradley-Terry 模型：给定 prompt x、被选中的回答 y_w 与被拒绝的回答 y_l，模型认为「y_w 优于 y_l」的概率由两个奖励分数之差经过 sigmoid 得到：

$$P(y_w \succ y_l \mid x) = \sigma(r_\theta(x, y_w) - r_\theta(x, y_l)),$$

其中 $\sigma(z) = 1/(1+e^{-z})$ 是 sigmoid，$r_\theta(\cdot)$ 是奖励模型输出。对应的负对数似然损失为

$$\mathcal{L}_{\text{BT}} = -\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l)) = \text{softplus}(-[r_\theta(x, y_w) - r_\theta(x, y_l)]).$$

直观理解：当「正例分数 − 负例分数」是很大的正数时，$-\log\sigma$ 接近 0，几乎没有损失；当正例分数反而更低时，损失迅速变大。视频里用 PyTorch 的 softplus 曲线演示了这一点，并给了一个数值例子：差值为 -0.1101 时损失约 0.74；差值足够大时损失硬性为 0。softplus 让「差距不够大」的样本也有平滑的梯度，而不是像 hinge 那样直接截断。

视频还介绍了另一种排序损失——边际排序损失（Margin Ranking Loss）：

$$\mathcal{L}_{\text{margin}} = \max(0, -y\,(x_1 - x_2) + \text{margin}),$$

其中 x_1 是第一条输入的分数、x_2 是第二条的分数，标签 $y = 1$ 表示「第一条应该排在前面」，若 x_1 确实大于 x_2 则损失为 0；`margin` 是可选间隔，默认 0。它和 Bradley-Terry 的区别是 hinge（硬截断）与 softplus（平滑）的差别；本讲训练使用 Bradley-Terry，但把 margin ranking loss 也实现出来做对照。

```
# r_pos / r_neg: 形状 (B, 1), chosen 与 rejected 的偏好分数
diff = r_pos - r_neg
loss_bt = -torch.nn.functional.logsigmoid(diff).mean()
# 等价写法: softplus(-diff) 或 BCEWithLogitsLoss(diff, ones)
```

Listing 19: Bradley-Terry 损失的核心计算（按视频讲解重写）

标签方向写反是最隐蔽的错误

Bradley-Terry 里「正例减负例」的顺序、MarginRankingLoss 里 $y = 1$ 的含义必须一致：chosen 在前、rejected 在后。方向写反时损失仍会下降（它会把模型训成「讨厌好回答」），但训练指标会异常。写完后先手动算一个 batch 的分数差，确认正例分数更高。

8.6 7.6 训练循环与评测：小数据只能验证管线

训练配置：300 步、batch size 8、上下文长度 256、2 层 encoder、2 个头、嵌入维度 128，使用 Bradley-Terry 损失，并复用 SFT 阶段的 tokenizer。每 25 步记录一次 loss；训练结束后进入评测循环：加载 checkpoint、eval 模式、batch 16，前向得到每条样本的 r_{pos} 与 r_{neg}，若 $r_{\text{pos}} > r_{\text{neg}}$ 则判对，统计准确率。

在小数据上（训练/测试各只有 8 条左右）结果大约是：训练集 75

奖励模型阶段的核心结论

奖励模型不生成文本，只给整段「指令 + 回答」打分；它的训练目标是让 chosen 的分数高于 rejected，而不是拟合某个绝对分数。小数据上「75% / 50%」的准确率只能说明管线跑通，不能说明 RM 有效——这正是下一阶段 PPO 需要大量数据与工程调优的原因。

8.7 7.7 与 Part 8 的衔接：进入强化学习

Part 7 结束后，视频立刻进入最后一部分（RLHF + PPO）的术语铺垫，给出的术语表包括：state（当前文本上下文）、action（下一个 token）、policy（策略，即语言模型）、reward（奖励，来自 RM）、value（价值函数）、advantage（优势）、GAE（广义优势估计）、KL 散度（约束新策略不要偏离参考模型太远）等。这些概念是 Part 8 实现 PPO 的前置知识，也说明了奖励模型的用途——它给出的标量分数正是 RL 阶段的 reward。

8.8 本章小结

1. 奖励建模是 RLHF 的第一步：用人类偏好比较数据训练一个对「(指令, 回答)」打分的模型，输出标量作为下一阶段 RL 的奖励信号。
2. 偏好数据用 chosen/rejected 成对组织（本讲用 Anthropic/hh-rlhf，训练集约 16.1 万行）；没有独立 prompt 字段时用空占位，换数据集时可替换。
3. 本讲实现 encoder-only 奖励模型：双向注意力可看全文，token+position embedding、堆叠 TransformerEncoderLayer、层归一化、标量线性头；参数可比语言模型小几个数量级。
4. 前向的关键是 padding mask 与 masked mean pooling：只对非填充 token 取平均得到 $B \times d$ 表示，再映射成 $B \times 1$ 分数。
5. 损失用 Bradley-Terry：$-\log \sigma(r_{\text{pos}} - r_{\text{neg}})$，等价于对负差值取 softplus；margin ranking loss（hinge 形式）作为对照。
6. 评测看「chosen 分数是否高于 rejected」的准确率；小数据上的数字只能验证管线，奖励模型与 RLHF 的收敛需要更大规模数据与算力。

9 Part 8 —基于 PPO 的 RLHF

这是整门课的最后一程：Part 6 我们用指令数据做了 SFT（监督微调），Part 7 用人类偏好数据训练了奖励模型（reward model, RM），Part 8 要把这两样东西接起来——把奖励模型当作环境给出的奖励，用 PPO（Proximal Policy Optimization）去优化语言模型，让它生成人类更喜欢的回答。这个阶段就是通常所说的 RLHF（Reinforcement Learning from Human Feedback）。

这一讲是纯代码实现：讲师在 `train_ppo.py`、`rollout.py`、`ppo_loss.py`、`eval_ppo.py` 里从零搭出一套最小但完整的 PPO-RLHF 训练循环，包括三个模型的加载与冻结、rollout 采样、上下文截断与 mask、token 级 log 概率、KL 奖励塑形、优势标准化、PPO 裁剪损失、训练日志与评测。读懂这套代码，你就拥有了理解所有现代 RLHF 实现（TRL、OpenRLHF 等）的骨架。

9.1 RLHF 三阶段与 PPO 的位置

先回看整个训练管线（图 **??**）：预训练（pretraining）得到基座模型；SFT 用「指令—回答」示范数据做监督微调，得到参考模型 π_{SFT}；奖励建模用人类偏好对训练一个打分模型 r_ϕ；最后 RLHF 用 r_ϕ 作为奖励，用 PPO 更新策略。

前两个阶段在本课程中已经完成。奖励建模用的是 Bradley–Terry 模型，损失为

$$\mathcal{L}_{\mathrm{RM}}(\phi) = -\,\mathbb{E}_{(x,\,y_w,\,y_l)}\Big[\log \sigma\big(r_\phi(x,y_w) - r_\phi(x,y_l)\big)\Big],$$

其中 x 是 prompt，y_w 是人类更喜欢的回答（chosen），y_l 是更差的回答（rejected），$r_\phi(x,y)$ 是奖励模型对「prompt 与回答」给出的标量分，σ 是 sigmoid。它要求好回答的分数高于坏回答。

现在进入 RLHF。它的优化目标不是「模仿某个回答」，而是「最大化奖励模型的期望打分，同时不要离参考模型太远」：

RLHF 的优化目标

$$\max_{\theta}\ \mathbb{E}_{x\sim\mathcal{D},\ y\sim\pi_\theta(\cdot|x)}\Big[r_\phi(x,y)\ -\ \beta\,\mathrm{KL}\big(\pi_\theta(\cdot\mid x)\,\|\,\pi_{\mathrm{ref}}(\cdot\mid x)\big)\Big],$$

其中 θ 是策略参数，$\pi_\theta(\cdot\mid x)$ 是当前策略（正在训练的 LLM），$\mathcal{D}$ 是 prompt 分布，r_ϕ 是冻结的奖励模型，π_{ref} 是冻结的参考模型（就是 SFT 模型），β 是 KL 惩罚系数。

为什么要加 KL？讲师在课上讲得很直白：RL 阶段的目标是迎合人类偏好，模型完全可能为了更高的奖励分而「跑偏」——输出风格变差、语言能力退化，甚至学会钻奖励模型的空子；更严重的是**灾难性遗忘**（catastrophic forgetting），即把 SFT 阶段学到的能力丢掉。KL 项的作用有两个：一是**度量**当前策略偏离 SFT 有多远，二是用这个偏离量去**惩罚**奖励，从而把更新限制在稳定范围内（这也就是 PPO 中「proximal」的含义之一）。

9.2 把语言生成写成强化学习问题

开始写代码之前，必须先把「LLM 逐 token 生成」翻译成 RL 语言。课上给出的这张术语表（图 **??**）是整讲的地基：

RL 概念	在本课程的 LLM 中对应什么
State s_t	当前文本上下文：prompt 加上到目前为止已生成的 token
Action a_t	下一个 token。本课程 toy 模型的词表大小为 8000，所以每一步是 8000 选 1
State–action (s_t, a_t)	某个具体上下文，以及在这个上下文里选择的具体下一个 token
Policy π_θ	LLM 给出的 next-token 概率分布，也是本阶段要更新的对象
Value $V(s_t)$	在上下文 s_t 下继续按当前策略生成，预期能拿到的总奖励预测
Reward r_t	标量分。RLHF 中通常只在回答结束时给一次：奖励模型的打分（本讲还在每个 token 上叠加 KL 惩罚）
Returns G_t	从第 t 步开始未来奖励的（折扣）和；它和 value 的差就是 value loss 的来源
Q-value $Q(s_t, a_t)$	在 s_t 下先选 token a_t、之后按策略继续的期望回报
Advantage	$A = Q - V$：选这个 token 比「该上下文的平均水平」好多少
KL divergence	当前策略与冻结参考模型在同一上下文下分布的距离，用来衡量偏离 SFT 的程度
Policy vs Ref	Policy 是不断更新的 RL 模型；Ref 是冻结的 SFT 模型，只用于 KL 约束

有几个数字要记住：本课程模型的词表大小是 8000；上下文窗口 `block_size` 是 256；所以一次前向的 logits 形状是「batch × 序列长度 × 8000」。这与普通语言模型训练看起来一样，区别在于：训练信号不再来自「下一个 token 的标签」，而来自奖励模型对整条回答的打分。

语言模型 RL 与经典 RL 的四个不同

(1) **动作空间巨大**：每一步要在整个词表（本讲 8000 个 token）上做 softmax；(2) **环境是确定性的**：给定上下文，下一个 token 由策略采样，没有外部模拟器；(3) **奖励稀疏、只在结尾**：生成完才由奖励模型打分，中间没有外部奖励，因此 credit assignment 要靠 value/returns 来做；(4) **回合长度可变**：每条回答的 token 数不同，所有损失与统计都必须用 mask 只统计「模型自己生成的 token」，不能把 prompt 与 padding 算进去。

9.3 训练器结构：三份权重与超参数

训练脚本是 `train_ppo.py`，入口 `main()` 先把超参数用 `argparse` 暴露出来（图 **??**）。这些默认值就是本讲实验的配置：训练步数 100、batch 大小 4、上下文窗口 256、回答长度 64、KL 系数 0.01、折扣因子 $\gamma = 1.0$、GAE 的 $\lambda = 0.95$、学习率 1×10^{-5}。

参数	取值	作用
`steps`	100（演示 50）	训练循环的步数
`batch_size`	4	每步采样的 prompt 数（记为 P），也是 PPO 的 batch
`block_size`	256	上下文窗口上限（prompt + 回答的总长度）
`resp_len`	64（演示 128）	每条回答最多生成的 token 数
`kl_coef`	0.01	KL 惩罚系数 β
`gamma`	1.0	折扣因子；只在结尾给奖励时取 1 最简单
`lam`	0.95	GAE 的 λ（参数保留；本讲示例实现做了简化，见「奖励塑形、KL 控制与优势估计」一节）
`lr`	1×10^{-5}	AdamW 学习率（RLHF 通常远小于 SFT）

接着加载模型。PPO-RLHF 需要三份权重（图 **??**）：

1. **Policy**：`PolicyWithValue`，由语言模型主干加一个线性 value head 组成。它从 Part 6 的 SFT 检查点初始化，是唯一需要训练的部分（主干 + value head）；
2. **Reference**：同样架构的 `PolicyWithValue`，复制 SFT 权重后设置 `requires_grad_(False)` 并切到 `eval()`，全程冻结，只为计算 KL；
3. **Reward model**：从 Part 7 的检查点加载，输入是格式化后的「prompt + 回答」，输出一个标量奖励；同样冻结、只做前向。Part 7 的奖励模型是 encoder-only 结构（双向注意力 + masked mean pooling + 标量头），参数量比语言模型小很多，所以多一份前向的代价可以接受。

对应的初始化代码大致如下（据课堂代码整理）：

```
policy = PolicyWithValue(vocab_size, block_size, n_layer, n_head, n_embd).to(device
    )
policy.lm.load_state_dict(ckpt["model"])        # 用 SFT 权重初始化语言模型

ref = PolicyWithValue(vocab_size, block_size, n_layer, n_head, n_embd).to(device)
ref.lm.load_state_dict(ckpt["model"])
for p_ in ref.parameters():
    p_.requires_grad_(False)                    # 参考模型全程冻结
ref.eval()

rm = RewardModel(vocab_size=..., block_size=...).to(device)
rm.load_state_dict(rcpt)
rm.eval()                                       # 奖励模型只做前向
```

Listing 20: 加载三份权重：policy（可训练）、ref（冻结）、reward model（冻结）

为什么 value head 要和 policy 共享主干？因为 critic 需要和 actor 一样理解语言；共享主干只多一个线性头，省下一整份大模型的显存与前向开销。代价是 policy 与 value 的梯度会互相影响，工程上通常用一个较小的 `vf_coef`（本讲 0.5）来平衡。

9.4 Rollout：采样、奖励与 token 级数据

训练循环的第一步是 rollout：用当前策略采样回答。图 **??** 是采样代码：从 Alpaca 数据集（与 SFT 相同的数据源）取 prompt，按 SFT 的对话格式拼好、去掉末尾的 `</s>`（因为后面要自己生成

回答），然后逐个 prompt 调用 `policy.generate`，最后把 prompt + 回答的完整 token 序列存起来。注意生成参数：`temperature=0.2`、`top_k=3`，即采样但仍偏保守；`max_new_tokens` 由 `resp_len` 控制。

拿到生成结果后，要对每条样本做三件事（图 **??**）：

1. **切分 prompt 与回答**：prompt 的 token 数就是边界，记作 `boundary`；从边界一直切到序列末尾，就是模型生成的全部回答 token。
2. **用奖励模型打分**：把回答解码成文本，再用 SFT 的格式化函数重新拼成「instruction + response」格式，喂给奖励模型，得到标量奖励 r；
3. **记录轨迹**：把（完整 token 序列，boundary，奖励）存进 `data` 列表，供后续构造 batch。

这里有一个关键细节：RLHF 的奖励是「整条回答一个分」，但 PPO 的损失是 token 级的。因此实现上要把这个分数放到回答的**最后一个生成 token** 上，其余 token 的奖励为 0（KL 惩罚除外，见「奖励塑形、KL 控制与优势估计」一节）。另外，回答长度不一，必须把一批样本 pad 到同一长度，并记录哪些位置是「模型生成」、哪些是「prompt」、哪些是「padding」——这就是动作掩码（action mask）的由来。

上下文截断：宁可裁 prompt，绝不裁回答。 课程的上下文窗口只有 256，而 prompt 可能很长。讲师用一张手绘图（图 **??**）说明了规则：如果 prompt 250 个 token、回答 10 个 token，一共 260 超过 256，那么从 260 里裁掉多出的部分——**只从 prompt 的左侧裁**，回答的 10 个 token 必须完整保留，于是变成「246 + 10」。

图 **??** 是对应的代码：先算这一批里最长的样本，并与 `block_size` 取最小值得到 `max_len`；对每条样本，`drop = L_full - L` 是要裁掉的数量，`b = max(0, boundary - drop)` 是裁剪后回答起点的位置；`seq[i, :L] = ids[-L:]` 只保留末尾 `L` 个 token（即从左边裁），不足部分用 `<pad>`（代码里是 token id 2）填充；`mask[i, b:L] = True` 标出回答 token；`rewards[i, L-1] = r_scalar` 把奖励放在回答最后一个位置上。

截断与 mask 的三个易错点

(1) 截断只能发生在 prompt 侧；如果连回答 token 一起裁掉，奖励就挂在了一个不完整的回答上，训练信号会错位。(2) pad 之后 mask 必须精确覆盖「回答」区间，prompt 和 padding 都不能算进 loss 或 value loss。(3) 奖励要放在回答的最后一个 token 上（`L-1`），而不是序列末尾的 padding 上——后者会让奖励位置错位一格。

9.5 从 token 到 log 概率：shift 标签与 gather

PPO 需要每个生成 token 的对数概率 $\log \pi_\theta(a_t \mid s_t)$。这一步和语言模型训练很像，但方向相反：训练时我们是「输入 token、预测下一个 token」，所以标签要右移一位——图 **??** 画的就是 `logits` 对应 t_1, t_2, t_3, t_4，而 `labels` 是 t_2, t_3, t_4。也就是说，位置 t 的 logits 用来预测位置 $t+1$ 的 token。

具体做法是：先对 logits 做 `log_softmax`，再用 `gather` 按标签索引取出对应 token 的 log 概率：

$$\log \pi_\theta(a_t \mid s_t) = \log_\mathrm{softmax}(\mathrm{logits}_t)\big[\, a_t \,\big],$$

其中 $\text{logits}_t \in \mathbb{R}^{8000}$ 是第 t 个位置对词表所有 token 的打分，log_softmax(·) 把打分变成对数概率，a_t 是实际生成的 token id，方括号表示按索引取值。以课上调试的数值为例：某个位置的标签是 203，对应位置的 log 概率是 −48.3569；另一个位置标签是 297，取出 −25.1141——这与手动从 8000 个对数概率里取第 203/297 个完全一致。

注意两个形状细节：序列长度是 159 时，右移后的标签只有 158 个位置，所以 log 概率也是 158；value 也是每个位置一个标量，同样要去掉最后一个位置变成 158。最后再把动作掩码作用上去，只保留回答 token：示例 batch 的回答 token 总数是 325（四个样本分别是 67、19、111、128 个回答 token）。此后所有的 policy loss、value loss、KL、奖励都在这个「长度 325 的扁平向量」上计算。

动作掩码（action mask）是 token 级 RLHF 的核心工程

语言模型生成里混杂了三类 token：prompt（用户给的）、回答（模型生成的）、padding（补齐的）。只有**回答 token** 是「策略的动作」，才有 policy loss、value、KL 与奖励。掩码一旦错位（例如长度 159 与 158 没对齐），损失会静默地算到错误位置，训练照样跑但学不到东西——这是 RLHF 实现里最常见、也最难 debug 的一类 bug。

9.6 奖励塑形、KL 控制与优势估计

现在每个回答 token 都有奖励模型给的（放在结尾的）分数，接下来要把它变成逐 token 的训练信号。第一步是计算 KL：用「当前策略 log 概率 - 参考模型 log 概率」作为单样本估计：

$$\widehat{\mathrm{KL}}_t = \log \pi_{\text{old}}(a_t \mid s_t) - \log \pi_{\text{ref}}(a_t \mid s_t),$$

其中 π_{old} 是 rollout 时（即更新前）的策略，π_{ref} 是冻结的 SFT 参考模型，a_t 是采样到的 token。这个估计就是「对数概率之差」，无偏但方差较大、也可能为负；它的好处是只需要两个前向，代价极小。实现上先算 `kl = (old_logp - ref_logp)`，再把它作为惩罚从奖励里减掉：

$$\tilde{r}_t = r_t - \beta\, \widehat{\mathrm{KL}}_t, \qquad r_t = \begin{cases} 0, & t < T, \\ r_\phi(x, y), & t = T, \end{cases}$$

其中 $\tilde{r}_t$ 是塑形后的逐 token 奖励，r_t 是原始奖励（只有最后一个生成 token T 才有奖励模型分数），β 是 KL 系数（代码里的 `kl_coef`，默认 0.01）。这一步在代码里就是 `shaped_r = rewards[:, 1:][act_mask] - args.kl_coef * kl`。

有了塑形奖励，就可以算回报与优势。这里要注意本讲实现的一个**有意简化**：代码注释明确写着，为了教学简洁，这里**不使用 GAE**，而是直接用「塑形奖励减 value」作为优势。因为本讲只在回答结尾给奖励、中间步没有奖励，塑形奖励可以直接当作 return 使用，于是

$$\hat{A}_t = \tilde{r}_t - V_\psi(s_t),$$

其中 $\hat{A}_t$ 是优势，$\tilde{r}_t$ 是上面塑形后的奖励，$V_\psi(s_t)$ 是 value head 对当前上下文的预测。为了让梯度尺度稳定，代码还会对整批优势做标准化：

$$\hat{A}_t \leftarrow \frac{\hat{A}_t - \text{mean}(\hat{A})}{\text{std}(\hat{A}) + \epsilon}, \qquad \epsilon = 10^{-6},$$

其中 $\text{mean}(\hat{A})$、$\text{std}(\hat{A})$ 是这一批 325 个优势的均值与标准差，ϵ 防止除零。

标准 PPO 的 GAE，以及本讲为什么可以先不用

标准 PPO 用广义优势估计（GAE）在「偏差」与「方差」之间权衡：

$$\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t), \qquad \hat{A}_t^{\mathrm{GAE}} = \sum_{l=0}^{T-t} (\gamma\lambda)^l \, \delta_{t+l},$$

其中 δ_t 是 TD 残差（这一步比 critic 预期好多少），γ 是折扣因子，λ 控制偏差—方差的权衡（$\lambda = 0$ 只看一步、$\lambda = 1$ 接近 Monte Carlo）。本讲的命令行里保留了 `--gamma 1.0` 与 `--lam 0.95` 两个参数，但示例实现为了教学简洁，在「只在结尾给奖励」的前提下直接用塑形奖励当 return，跳过了 GAE 的循环计算。如果你要把它改成标准 PPO（例如带中间奖励、工具回调），把这里换成 GAE 即可，公式与上面的完全一致。

9.7 PPO 目标与损失函数

PPO 的核心是「不要一次更新太远」。它用新旧策略在同一动作上的概率比来限制步长：

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\mathrm{old}}(a_t \mid s_t)} = \exp\Big(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\mathrm{old}}(a_t \mid s_t)\Big),$$

其中 ρ_t 是重要性比率，π_θ 是正在更新的策略，π_{old} 是 rollout 时的策略，a_t 是实际采到的 token。然后用裁剪的代理目标：

$$\mathcal{L}^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t \hat{A}_t,\ \mathrm{clip}(\rho_t,\, 1-\epsilon,\, 1+\epsilon)\,\hat{A}_t\big)\Big],$$

其中 $\hat{A}_t$ 是优势，ϵ 是裁剪范围（本讲 `clip_ratio=0.2`）。直觉：优势为正时，比率涨到 $1+\epsilon$ 就不再给额外奖励；优势为负时，比率跌到 $1-\epsilon$ 就不再额外惩罚——超过这个范围，策略离旧策略太远，继续优化也不可靠。

完整的损失还要加上 value loss 和可选的熵奖励。图 **??** 是 `ppo_loss.py` 的完整源码：策略损失是负的裁剪代理目标；value loss 是 value 与 returns 的均方误差（代码注释说「clip optional，这里用简单 MSE」）；熵用 $-\overline{\log \pi}$ 近似；最后

$$\mathcal{L} = \mathcal{L}_{\mathrm{policy}} + c_v\, \mathcal{L}^{\mathrm{VF}} - c_e\, \mathcal{H},$$

其中 $\mathcal{L}_{\mathrm{policy}} = -\mathcal{L}^{\mathrm{CLIP}}$ 是策略损失，c_v 是 value 系数（本讲 `vf_coef=0.5`），$\mathcal{L}^{\mathrm{VF}} = \frac{1}{N}\sum_t (\hat{R}_t - V_\psi(s_t))^2$ 是 value loss（$N = 325$ 为回答 token 数，$\hat{R}_t$ 是上面算出的 return），c_e 是熵系数（本讲 `ent_coef=0.0`，即不加熵），$\mathcal{H}$ 是策略熵——加熵可以鼓励探索，防止模型过早变得过于确定。

把上面代码翻译成伪代码，就是下面这个关键片段（据课堂代码整理）：

```
def ppo_losses(new_logp, old_logp, adv, new_values, old_values, returns,
               clip_ratio=0.2, vf_coef=0.5, ent_coef=0.0):
    # 1) 策略损失：重要性比率 + 裁剪
    ratio = torch.exp(new_logp - old_logp)
    unclipped = ratio * adv
    clipped = torch.clamp(ratio, 1.0 - clip_ratio, 1.0 + clip_ratio) * adv
    policy_loss = -torch.mean(torch.min(unclipped, clipped))

    # 2) value 损失：value head 回归 returns
    value_loss = F.mse_loss(new_values, returns)

```

```
    # 3) 熵奖励：用 -logp 的均值近似策略熵
    entropy = -new_logp.mean()

    # 4) 监控用的近似 KL（不参与总损失）
    approx_kl = torch.mean(old_logp - new_logp)

    total = policy_loss + vf_coef * value_loss - ent_coef * entropy
    return PPOLossOut(policy_loss, value_loss, entropy, approx_kl, total)
```

Listing 21: PPO 损失函数的核心实现（据 ppo_loss.py 课堂代码整理）

注意 old_values 在函数签名里出现，但在这个简化实现里并没有进入损失（value loss 直接用 returns 当目标）；approx_kl 只用于记录，KL 惩罚已经通过「奖励塑形、KL 控制与优势估计」一节的奖励塑形进入训练。另外，demo 里这一段更新**只跑一次**——代码注释明确写着「This step is done multiple times per batch in practice」，标准 PPO 会对同一批数据做多轮（常见 4–8 轮）mini-batch 更新，这也是 PPO 样本效率的来源。

维度	本讲的示例实现	标准 PPO / 生产实现
优势估计	直接用塑形奖励当 return，adv = shaped_r - old_values（无 GAE）	用 GAE 累积 TD 残差，γ、λ 可调
KL 惩罚位置	从奖励里逐 token 扣除（reward shaping）	也可作为独立 loss 项；常配自适应 β
value 裁剪	注释「optional」，直接用 MSE	常见 value clipping 或双 value 网络
每批更新轮数	单次通过（demo 简化）	同一批数据多轮 mini-batch 更新

9.8 训练与评测：日志、KL 变化与 demo 的局限

训练循环的结构是：采样 rollout → 计算 KL/奖励/returns/优势 → 标准化优势 → 一次 PPO 更新（反向传播 + 梯度裁剪）→ 记录指标。课程把 50 步训练跑了一遍（图 **??**）：每 10 步打印一次 loss、value loss、KL_move（这一步策略相对上一步的移动量）与 KL_ref（策略相对冻结参考模型的累计偏离）。

从日志里能读到三件事：

- **总损失与 value loss 都在下降**：说明策略在改进、critic 对 returns 的预测越来越准（对应讲师说的「value 会越来越好地估计未来奖励」）；
- **KL_move 很小、KL_ref 逐步变大**：前者说明单步更新被 PPO 的裁剪与学习率控制住了，后者说明策略确实在慢慢偏离 SFT 参考模型——这正是要看 KL 的原因；
- **评测的平均 RM 奖励是负的（-1.2351）**：在这个 toy 设置下，奖励模型本身就不是很强，训练目标也只是「尽量拿高分」，所以这个数字只能作为对照，不能当作「模型变好」的证据。

评测脚本 eval_ppo.py（图 **??**）的做法很直接：加载训练后的 policy 和冻结的 ref，对同一批示例 prompt 分别生成回答并解码，再用奖励模型对「格式化后的 prompt + 回答」打分并取平均。

讲师用它对比了训练前后的回答：训练后同一个 prompt 的回答「变成了别的东西」——说明策略确实偏离了 SFT 初始状态；但在这个只有 16 个提示词、两层 Transformer 的 toy 设置里，这种偏离并没有实际意义。

demo 能跑通不等于 RLHF 有效

讲师在结尾特别强调：这个 demo 只有 16 个数据点、一个两层的小模型，奖励模型也不是最好的，所以「这些都不会很有意义，你实际上需要大量的数据」。真正做 RLHF 时要注意：(1) 奖励模型只是人类偏好的**代理**，策略会学会骗它（reward hacking），表现为回答变长、堆砌讨好词、模板化；(2) 评估不能只看训练奖励，必须看留出偏好集、人工评估与下游指标；(3) 大模型 RLHF 的显存与吞吐开销远高于 SFT，rollout 与四个模型的前向是主要瓶颈。

把这一讲的实践要点浓缩成一张 checklist：

- 三份权重：policy（含 value head，训练）、ref（冻结）、reward model（冻结）；
- rollout 记录完整 token 序列、boundary、奖励；上下文超限时只裁 prompt；
- 标签右移一位后用 `log_softmax + gather` 取 log 概率；所有损失只统计回答 token；
- KL 用 `old_logp - ref_logp` 估计，并从奖励中扣除（`kl_coef`）；
- 优势做标准化、梯度裁剪（1.0）、学习率取小（1e-5）；
- demo 单次更新即可跑通；标准 PPO 对同一批数据多轮更新；
- 监控 `KL_move` 与 `KL_ref`，警惕 reward hacking 与灾难性遗忘。

9.9 本章小结

1. RLHF 在 SFT 与奖励模型之后，用 PPO 优化「奖励模型分数 − KL 惩罚」这一目标；KL 的作用是限制策略偏离 SFT 参考模型、抑制能力退化与灾难性遗忘。
2. 语言生成就是 MDP：state 是上下文、action 是下一个 token、reward 来自奖励模型（本讲只在结尾给一次）；一个 PPO-RLHF 训练器需要 policy（含 value head）、冻结的 ref 与冻结的 reward model 三份权重。
3. rollout 要把 prompt 与回答切分、把奖励放在回答最后一个 token 上；上下文超限时只从 prompt 侧裁剪，并用 action mask 精确标出回答 token——mask 错位是最常见的 silent bug。
4. 本讲用「shift 标签 + log_softmax + gather」得到 token 级 log 概率；用 `old_logp - ref_logp` 估计 KL 并从奖励中扣除；优势用「塑形奖励 − value」并在批内标准化（示例实现省略了 GAE，但保留 γ, λ 参数，改起来只差一个循环）。
5. PPO 用重要性比率与裁剪（$\epsilon = 0.2$）限制更新幅度；总损失 = 策略损失 + c_v· value loss − c_e· entropy（本讲 $c_v = 0.5$、$c_e = 0$），配合梯度裁剪 1.0 与小学习率 1e-5 保证稳定。
6. 评估不能只看训练奖励：本讲 demo 的 `Avg RM reward`（-1.2351）只说明策略在逼近一个并不完美的奖励模型；真实 RLHF 需要大量数据、独立评估与对 reward hacking 的防范。

10 总结与延伸

这门 6 小时课程的主线只有一句话：**把「一段文本」一步步变成一个能按人类偏好回答问题的 LLM，并亲手写出每一步的 PyTorch 实现**。九个 Part 不是九个孤立知识点，而是一条严格的递进链：

一条主线串起九个 Part

文本/分词 → Transformer 架构 → 预训练一个小模型 → 架构现代化（RMSNorm/RoPE/SwiGLU/滑窗/KV cache）→ 扩展规模（混合精度/梯度累积/并行）→ MoE 稀疏化 → SFT 对齐指令 → 奖励建模 → PPO RLHF。

10.1 五个最值得带走的结论

1. **LLM 的每一层都可以被写出来**。从 byte-level tokenizer、带因果掩码的缩放点积注意力、多头拼接、残差 + LayerNorm，到交叉熵与反向传播，课程全部用纯 PyTorch 实现；理解“张量形状如何流动”比背公式更重要。

2. **现代架构的每个改动都在解决一个具体问题**。RMSNorm 去掉均值中心化以省算力；RoPE 把相对位置编码进 Q/K 旋转；SwiGLU 用门控提升 FFN 表达力；滑动窗口与 KV cache 共同决定长上下文与推理成本。判断一个改动是否值得，要回到“它解决什么、代价是什么”。

3. **训练工程决定能不能跑起来**。混合精度（bf16/fp16 + loss scaling）、梯度累积（用小显存模拟大 batch）、梯度裁剪与学习率调度，是把“数学上成立”变成“工程上收敛”的关键；显存账要按参数 + 梯度 + 优化器状态 + 激活分开算。

4. **MoE 是“用稀疏换容量”**。top-k gating 让每个 token 只走少数专家，参数量上去了但计算量可控；代价是路由/负载均衡、通信、训练不稳定与推理复杂度。它不是一个免费的放大按钮。

5. **对齐是“三阶段流水线”，不是单个算法**。SFT 教格式与指令遵循；奖励模型把人类偏好压缩成一个标量；PPO 用奖励 + KL 约束做策略优化。每个阶段的目标、数据与失败模式都不同，混在一起讲就会得出错误结论。

10.2 把课程接回你的项目与面试

- **SFT 与 loss mask**：课程里 chat template + assistant-only loss 的实现，可以直接对照你在 Code-R1/Search-R1 里的轨迹构造与训练数据组织；被问“SFT 为什么只算 assistant token”时，从 mask 的构造讲起最清楚。
- **奖励模型与 reward hacking**：课程对 Bradley–Terry 偏好损失、标量头、校准的讲解，可以补上“为什么过程奖励/奖励模型会被钻空子”的理论背景；结合你 verifier 门控失败（gate 18.2%、LCB 24.8）与手写 PRM 被 hack 的经历，是一条完整叙事。
- **PPO 与 GRPO**：课程给出 PPO 的 ratio/clip/advantage/KL 与 GAE 的完整推导；对照你项目里的 GRPO（组内优势、砍 critic、KL 惩罚），可以回答“为什么可验证任务用 GRPO、长程稀疏任务仍要考虑 PPO/value model”。

- **MoE 与训练系统**：专家路由/负载均衡、通信开销与并行策略，和你在 09-SWE-agent 里做的 rollout/训练解耦、显存与吞吐优化是同一类工程问题；面试聊“训练系统”时可以从这里借术语。
- **从零实现的价值**：面试官问“你知道 Transformer 内部长什么样吗”时，能说出每层的张量形状、参数量与计算量，比复述论文更有说服力。

10.3 如果继续深入：三条延伸线

1. **把 tiny LLM 放大**：用课程代码在自己的数据上做一次小规模预训练（1–2 张卡），记录 loss 曲线、吞吐与显存，形成“训练工程”的第一手数据。
2. **把对齐做完整**：在 SFT 模型上训练奖励模型，再跑 PPO/GRPO，对比“纯 SFT vs SFT+RM+PPO”的差异，观察 KL、reward 与长度变化——这是理解 RLHF 稳定性最直接的方式。
3. **把 MoE 讲清楚**：实现一个最小 top-2 MoE block，测量专家利用率、负载不均衡与吞吐，回答“MoE 什么时候真的划算”。

学习建议

第一遍跟着视频敲代码、跑通形状；第二遍回到本讲义，按“动机 → 机制 → 代码/公式 → 工程坑”复述每个 Part；第三遍只读各 Part 的“本章小结”和本节的五个结论，用它们回答面试里的“为什么”。